

PROBA: Threshold-Authorized and Privacy-Preserving Blockchain-Powered Helios

Yarkın Doröz , Sermin Kocaman , Köksal Muş , Fatih Sulak , and Oğuz Yayla 

Abstract—Helios is a pioneering web-based, end-to-end verifiable voting system intended for low-coercion elections. Replacing its centralized bulletin board with a blockchain can improve data replication and public auditability, but a direct implementation such as the blockchain-powered Helios construction of Perez and Ceesay leaves three deployment concerns: a single contract owner controls wallet authorization, the registration authority learns the identity–wallet association, and voters may be required to obtain public-chain tokens. We present PROBA, a focused extension that replaces the on-chain eligible-wallet list with threshold-issued anonymous credentials and deploys the bulletin-board layer on a permissioned Ethereum-compatible network. In this architecture, credentials are blindly issued to an election-specific wallet public key by issuers that each authenticate the voter independently. During the casting phase, the voter proves possession of the credential and signs the content identifier of a well-formed encrypted ballot, allowing the smart contract to natively enforce an explicit last-valid-ballot rule. We give a role-specific threat model and modular game-based arguments for eligibility, controlled re-voting, ballot privacy, registration-to-wallet unlinkability, and individual and universal verifiability. While these guarantees rely on explicit assumptions regarding the issuer, trustee, validator, storage, and communication channels, PROBA explicitly does not claim coercion resistance, receipt-freeness, or software independence. We implement PROBA on BLS12-381 together with an EIP-2537 smart contract that enforces every acceptance check on-chain. A voter prepares a five-candidate ballot, including the credential showing, in 4.83 ms, and a verifier checks it off-chain in 5.72 ms; the credential layer contributes 0.20 ms and 0.93 ms, respectively. On-chain acceptance costs about 0.89 million gas per ballot, 82.6% of which is proof verification. This is about 12 times the record-only transaction of the baseline; it is feasible when a permissioned network prices voter transactions at zero and rate-limits re-voting per wallet. Complete elections with up to 1,000 voters and re-voting produced correct and independently re-verified tallies.

Index Terms—Anonymous credentials, blockchain bulletin board, Helios, Internet voting, threshold cryptography, verifiability.

I. INTRODUCTION

Web-based voting platforms seek to make participation independent of polling-place access while providing faster tallying and a public audit trail. Their deployment ranges from

state-operated Internet voting, such as the Estonian model discussed in recent comparative studies, to lower-stakes cryptographic platforms such as Helios and Belenios for universities, professional associations, companies, and civil-society organizations [1]–[3]. The security problem is not merely to encrypt votes: a complete system must authenticate eligible voters, prevent unauthorized or duplicate influence, protect ballot secrecy, publish sufficient evidence for verification, and remain available despite faults or malicious behavior [4]–[6].

Helios is a prominent web-based open-audit system for low-coercion elections [2], [7]. It combines exponential ElGamal, ballot-validity proofs, a Benaloh-style challenge for cast-as-intended checking, public encrypted ballots, and threshold decryption. Helios therefore offers individual and universal verification without revealing individual plaintext votes. [8]–[10]. While these properties establish a robust foundation for end-to-end verifiability, the system’s security model inherently operates within specific operational boundaries. Specifically, Helios is not designed to be coercion-resistant or receipt-free; the voter-facing software remains security-critical, and the system does not provide a voter-verifiable paper record or another independent physical record. Furthermore, formal analyses have identified ballot-secrecy, ballot-malleability, and re-voting concerns in variants of Helios [8]–[10]. Rather than attempting to address all of these broader limitations, PROBA targets a narrower question: how to add a replicated public bulletin-board realization without introducing centralized wallet authorization and identity-wallet linkage.

A blockchain is one possible implementation of a public bulletin board, not a prerequisite for end-to-end verifiability. Peered and replicated bulletin boards were studied before blockchain-based voting, and complete distributed systems such as D-DEMOS demonstrate that fault-tolerant publication can be achieved without a cryptocurrency ledger [11], [12]. Blockchain deployments are attractive when an organization already operates an authenticated validator consortium and wishes to reuse deterministic finality, append-only records, and Ethereum-compatible smart contracts. These benefits are conditional: a permissioned chain redistributes trust among validators rather than eliminating it, and its safety and liveness depend on the consensus threshold.

Perez and Ceesay’s blockchain-powered Helios construction [13] stores encrypted ballots and their proofs in IPFS and places the resulting content identifiers on Ethereum. Before voting, a contract owner writes the eligible wallet addresses to the contract. This concrete authorization mechanism creates three deployment-level concerns.

P1. Centralized wallet authorization. A single contract owner can omit an eligible address, insert an ineligible

Y. Doröz is with Nvidia Corporation, CA 95051, USA, yarkin-doroz@gmail.com.

K. Muş are with the Department of Electrical and Computer Engineering, Worcester Polytechnic Institute, Worcester, USA, kmus@wpi.edu.

S. Kocaman is with the Department of Cryptography, Institute of Applied Mathematics, Middle East Technical University, Ankara, Türkiye, and also with FAME Crypt, Ankara, Turkey, sermin.cakin@gmail.com.

F. Sulak is with the Department of Mathematics, Atılım University, Ankara, Türkiye, fatih.sulak@atilim.edu.tr.

O. Yayla is with the Department of Cryptography, Institute of Applied Mathematics, Middle East Technical University, Ankara, Türkiye, oguz@metu.edu.tr.

address, or delay publication of the authorization list.

P2. Registration-to-wallet linkability. The party that authenticates a voter receives that voter’s wallet address. Although the ballot remains encrypted, this reveals which public blockchain account belongs to which person and permits linkage to all observable activity of that account.

P3. Voter-facing public-chain cost. A voter may need cryptocurrency and must accept volatile public-chain fees. This is a participation and deployment barrier, not a cryptographic failure.

Among these deployment barriers, the second concern requires nuanced clarification. It is deliberately not described as a direct break of ballot secrecy: learning a voter’s wallet does not by itself reveal the plaintext vote under secure encryption. The property addressed here is the narrower but operationally important separation between civil identity and the public election wallet. Network-layer metadata can still recreate that link unless submissions use an anonymity-preserving channel or the observer model excludes such metadata.

A. Contributions

This paper makes the following contributions.

- **Threshold-authorized wallet registration.** We replace the preloaded on-chain wallet list with a threshold-issued anonymous credential on the election identifier and the voter’s wallet public key. Fewer than the issuance threshold cannot create a valid credential, while no single contract owner decides eligibility; because every issuer authenticates the voter independently, the registration service cannot authorize a wallet on its own either.
- **Blind registration with explicit scope.** The issuers authenticate the voter but sign a blinded wallet key. The resulting guarantee is registration-to-wallet unlinkability under the blindness and unlinkability of the credential system and under the stated communication assumption. Ballots sent from the same wallet remain intentionally linkable so that the contract can enforce re-voting.
- **Complete ballot-acceptance and re-voting rules.** The protocol binds the credential, election identifier, wallet public key, ballot content identifier, digital signature, ballot-validity proofs, and a monotone sequence number. The contract retains only the latest valid ballot for each authorized wallet and bounds the number and frequency of re-votes per wallet, so that zero-priced transactions cannot be used to exhaust the ledger.
- **Role-specific security model.** We distinguish credential issuers, decryption trustees, validators, storage nodes, voters, and auditors. Security claims are reduced modularly to the properties of the employed primitives and to explicit corruption and availability thresholds.
- **Implementation and evaluation.** We implement the protocol on BLS12-381 with a smart contract that verifies checks C1–C5 through the EIP-2537 precompiles. We report primitive, per-role, and scaling costs, the gas of every acceptance check compared with the Perez–Ceesay baseline, differential tests of malformed ballots,

and complete elections with up to 1,000 voters, and we state explicitly which quantities (IBFT latency and wide-area IPFS) are not measured.

B. Scope

PROBA is intended for low-coercion organizational elections in which voters can complete a registration interaction before voting. It does not provide coercion resistance, receipt freeness, protection against a fully compromised voter device, or software independence. A permissioned blockchain is used as a concrete replicated bulletin board; the protocol can be adapted to another bulletin-board realization satisfying the publication properties in Section III.

C. Organization

Section II reviews the directly relevant literature and establishes the research gap. Section III defines the system, threat model, notation, security objectives, and cryptographic interfaces. Section IV formalizes Helios and the Perez–Ceesay baseline. Section V presents PROBA. Sections VI and VII give the security analysis and the implementation and performance evaluation, respectively. Section VIII states the limitations and deployment assumptions, and Section IX concludes.

II. RELATED WORK AND RESEARCH GAP

A. Helios and its security boundary

Helios uses homomorphic tallying and public proofs to provide cast-as-intended, recorded-as-cast, and tallied-as-recorded evidence [2], [6], [7]. Its real-world use has mainly involved university, professional-association, and organizational elections. Beyond the early university and ACM deployments, the IACR continued to use Helios-derived electronic voting for recurring association elections during the last decade; the maintained Helios service also remains available for organizational elections [14]–[17]. More broadly, recent end-to-end verifiable online-voting trials, including the 2022 Durga Puja deployment reported in 2024, show continuing practical interest in Helios-style public evidence even when a different protocol realization is used [18]. The intended low-coercion scope is essential: Helios is neither coercion-resistant nor receipt-free, and its browser software remains security-critical. Recent surveys continue to distinguish these limitations from end-to-end verifiability [1], [5].

Formal analyses have identified ballot-secrecy and ballot-independence weaknesses in Helios variants [8]–[10]. A representative *ballot-copying* attack proceeds when an adversary copies or transforms an honest voter’s encrypted ballot and submits the related ciphertext as another ballot. If the protocol does not bind the proof and ciphertext to the submitting voter and election context, the copied ballot can create correlated tally information or otherwise violate ballot independence. In PROBA, the ballot proof is a proof of knowledge of the encryption randomness for a statement containing $(\text{eid}, \text{pk}_v, \text{seq}_v, \text{ct}_v)$, and the resulting content identifier is signed by the same wallet. Consequently, a proof copied to another wallet or sequence does not verify, while generating

a fresh proof for the copied ciphertext requires the corresponding encryption witness. This binding addresses replay and copying at the authorization layer; it does not convert Helios into a coercion-resistant system.

Helios-inspired systems modify other components. Belenios strengthens the authentication and verification architecture [3]; Ordinos considers tally hiding [19]; Apollo adds recovery from vote manipulation [20]; and LWE-based variants explore post-quantum alternatives [21]. These systems are relevant context, but they are not direct baselines for the present authorization-layer contribution.

B. Anonymity and ballot privacy

Ballot privacy and voter anonymity are related but distinct. Ballot privacy hides the plaintext choice represented by a ciphertext, whereas identity anonymity or registration-to-casting unlinkability hides which public election pseudonym belongs to a civil identity. Homomorphic encryption can preserve ballot privacy even when the registrar knows the voter's wallet; conversely, a blind-signature or anonymous-credential mechanism can hide the identity-wallet association while leaving network metadata or a poorly encrypted ballot exposed [22], [23].

Prior work seeking both sender anonymity and ballot privacy targets a stronger joint goal than the original Perez-Ceasay authorization layer. PROBA separates these notions explicitly: it provides Helios-derived ballot privacy and registration-to-wallet unlinkability under Assumption 1, while network-level sender anonymity depends on the required anonymous communication channel. Ballots submitted from the same election wallet remain intentionally linkable to implement controlled re-voting.

C. Distributed bulletin boards and blockchain voting

A public bulletin board is an abstract functionality that gives parties a consistent, append-only view of election evidence. Peered bulletin boards and distributed voting systems provide this functionality through replicated protocols with explicit fault thresholds [11], [12]. Blockchain voting is one concrete realization of this layer, not a prerequisite for end-to-end verifiability. Surveys published since 2020 continue to identify identity management, privacy, scalability, consensus trust, and deployment cost as open problems [4], [5], [24]–[26]. Recent proposals include off-chain zero-knowledge processing, ElectAnon, EtherVote, and vSPACE [27]–[30]; their security models and ballot semantics differ substantially.

Perez and Ceasay [13] provide the closest baseline. Their design moves Helios audit material to IPFS and Ethereum, but eligibility is represented by wallet addresses inserted by a contract owner. PROBA preserves the Helios ballot and tally logic while replacing that authorization method with threshold-issued anonymous credentials and replacing a public fee market with an authenticated validator consortium.

III. SYSTEM MODEL, REQUIREMENTS, AND PRELIMINARIES

A. Entities and trust distribution

The protocol contains the following roles. A physical organization may operate more than one role, but the assumptions are stated per role.

- **Election organizer.** Defines the election identifier eid , ballot, a pre-registration window $(T_{reg}^{open}, T_{reg}^{close}]$, a voting window $(T_{vote}^{open}, T_{vote}^{close}]$, a credential-expiry time T_{cred} , issuer threshold t_i , decryption threshold t_d , and validator configuration.
- **Eligibility registry and registration service.** Maintains the authoritative list of eligible civil identities and an auditable election-specific issuance state. It reserves one pending request per identity and issues one-use authorization tokens bound to blinded requests, but it does not learn the unblinded wallet public key and cannot authorize a wallet on its own.
- **Credential issuers** $I_1, \dots, I_{n_i}$. Each issuer authenticates the civil identity independently through its own enrollment channel and keeps a local record that permits at most one share per identity and election. After authentication, at least t_i issuers provide partial blind credential shares.
- **Decryption trustees** $D_1, \dots, D_{n_d}$. At least t_d trustees produce publicly verifiable partial decryptions of the aggregate ciphertext.
- **Voters.** Each voter generates an election wallet key pair (sk_v, pk_v) , obtains one credential, creates a ballot, and may replace that ballot using a larger sequence number.
- **Validator consortium.** Executes the smart contract and provides a consistent append-only view under the IBFT fault assumption [31], [32].
- **IPFS pinning service.** Stores and replicates ballot objects. Content addressing protects integrity, while availability requires independent pinning and archival commitments.
- **Auditors.** Download the ledger and IPFS objects and independently verify the accepted-ballot set, proofs, aggregate ciphertext, partial decryptions, and result.

Assumption 1 (Role-specific adversary model). *The adversary is probabilistic polynomial time and may control arbitrary voters, $f_i < t_i$ credential issuers, fewer than t_d decryption trustees, and at most the Byzantine-validator bound supported by the configured IBFT network. It may also control the registration service, which can then deny or delay registration; its effect on identity-level uniqueness is stated in Proposition 15. The adversary cannot authenticate as an uncorrupted identity to an honest issuer (authentication soundness). If all issuers rely on the same identity provider, that provider is a single point of trust for eligibility and must be counted as such. Honest voters authenticate to issuers over confidential authenticated channels. Registration and ballot submission by an honest voter are carried over anonymity-preserving communication channels that prevent the adversary from correlating issuance and casting sessions through source addresses,*

TABLE I
FOCUSED COMPARISON OF THE SYSTEMS RELEVANT TO THE CONTRIBUTION. “CONDITIONAL” MEANS THAT THE PROPERTY DEPENDS ON THE STATED CORRUPTION AND COMMUNICATION ASSUMPTIONS.

Feature	Helios	Perez-Ceasay blockchain-powered Helios	PROBA
Bulletin board	Central web service	Ethereum references to IPFS objects	Permissioned Ethereum-compatible ledger referencing pinned IPFS objects
Authorization	Helios voter account / roll	Contract owner preloads eligible wallet addresses	Threshold issuers, each authenticating the voter independently, blindly issue an election-specific credential on a wallet key
Identity-wallet separation	No blockchain wallet in the original design	No: registrar receives identity and wallet address	Conditional: blindness/unlinkability plus the communication assumption
Ballot-copying resistance	Depends on the Helios variant and proof binding	Not made explicit in the baseline	Ballot proof and wallet signature are bound to election, wallet, sequence, and ciphertext
Ballot well-formedness	Helios ZKPs	Intended to be inherited; enforcement must be explicit	Verified before acceptance and again before tally inclusion
Re-voting rule	Helios-specific latest-ballot semantics	Not fully specified in the baseline	Monotone sequence number; latest valid ballot per wallet
Trust distribution	Election server and trustees	Contract owner, public-chain consensus, trustees, IPFS	Issuer threshold, registration-state consistency, trustee threshold, validator threshold, and ledger-history retention
Voter transaction fee	None at protocol level	Public-chain fee may be voter-facing	Consortium policy can subsidize or set zero-priced transactions
Coercion resistance / receipt freeness	No / no	No / no	No / no

timing, or browser identifiers. This is a required deployment assumption for the registration-to-wallet unlinkability theorem, not a property supplied by the credential primitive itself. The public client, ledger, and IPFS data are fully observable. Denial of service against all issuers, all pinning providers, or the whole network is outside the guaranteed availability envelope.

The model separates safety from liveness. Fewer than t_i issuers cannot forge an authorization credential, but registration remains available only if at least t_i responsive issuers and a responsive registration service exist. Ballot secrecy holds while fewer than t_d trustees collude, whereas tally availability requires at least t_d responsive trustees. The validator consortium redistributes trust: it does not eliminate the possibility of collusion beyond the consensus threshold.

B. Formal deployment concerns and target properties

Definition 2 (Centralized authorization misbehavior). A wallet-authorization mechanism has centralized-authorization misbehavior risk if a single administrator can cause an ineligible wallet to be accepted, an eligible wallet to be rejected, or the authorization set to be delayed, without producing publicly verifiable evidence attributable to a threshold of independent authorities.

Definition 3 (Registration-to-wallet linkability). Let an issuance transcript contain the authenticated identity information observed during registration, and let a casting tran-

script contain the public wallet and ballot transaction. The mechanism is registration-to-wallet unlinkable if no efficient adversary satisfying Assumption 1 can distinguish which of two honestly registered voters owns a challenge wallet with more than negligible advantage. This property does not hide that two ballots originate from the same wallet.

Definition 4 (Voter-facing transaction-cost barrier). A deployment has a voter-facing transaction-cost barrier if casting requires the voter to acquire a scarce external payment token or pay an unpredictable public-network fee. PROBA addresses this operational property by allowing the consortium to subsidize or configure transaction pricing; it does not claim that operating the infrastructure is cost-free.

Definition 5 (Immutable and available ballot publication). A ballot publication layer is immutable if an accepted record cannot be replaced or removed without violating the ledger safety assumption or finding a collision in the content-addressing hash. It is available through the audit period if at least one honest validator or archive retains the ledger history, which in PROBA contains every accepted object as calldata, or if at least one honest pinning provider retains every accepted object. Content addressing alone provides integrity, not availability.

Definition 6 (Privacy-preserving aggregation). An aggregation mechanism is privacy preserving if valid ciphertexts can be combined and the aggregate can be threshold-decrypted without revealing any individual plaintext ballot beyond in-

TABLE II
PRINCIPAL NOTATION.

Symbol	Meaning	Symbol	Meaning
λ	Security parameter	eid	Unique election identifier
n_v, n_c	Numbers of voters and candidates	n_i, t_i	Number and threshold of credential issuers
n_d, t_d	Total number and threshold of decryption trustees	n_b, f	Validators and tolerated Byzantine validators
pk_v, sk_v	Election wallet verification/signing keys	seq_v	Monotone ballot sequence number
$T_{reg}^{open}, T_{reg}^{close}$	Pre-registration window	$T_{vote}^{open}, T_{vote}^{close}, T_{cred}$	Voting window and credential expiry
mpk	Election encryption public key	mvk	Aggregate credential verification key
ct_v	Encrypted ballot vector	π_v^{bal}	Ballot-validity and selection proofs
σ_v	Aggregate anonymous credential	π_v^{cred}	Re-randomized credential showing (h'_v, s'_v)
cid _v	IPFS content identifier	τ_v	Wallet signature on the election ballot statement
Latest[pk]	Latest accepted ballot record for a wallet	RegDB[ID]	Election-specific issuance status for a civil identity
S_{max}, Δ_{min}	Per-wallet cap on the sequence number and minimum spacing of accepted ballots	f_i	Number of corrupted issuers, $f_i < t_i$

formation implied by the published tally. In PROBA this property is conditional on IND-CPA encryption, valid-ballot zero knowledge, and corruption of fewer than the decryption threshold.

Definition 7 (Threshold issuance). *A credential system has threshold issuance with parameters (n_i, t_i) if a valid aggregate credential requires shares from at least t_i distinct issuers, while fewer than t_i issuers cannot create a new valid credential except with negligible probability. It has one-more threshold issuance if an adversary that corrupts $f_i < t_i$ issuers and obtains blind shares from honest issuers on requests of its choice cannot output valid credentials on more distinct attributes than the number of requests that received shares from at least $t_i - f_i$ honest issuers.*

Definition 8 (Unlinkable randomized credential showing). *A credential showing is unlinkable if independently randomized showings generated from the same valid credential cannot be linked to each other or to the corresponding issuance transcript, except through attributes deliberately disclosed by the protocol. In PROBA the disclosed wallet key intentionally links re-votes from the same wallet.*

Definition 9 (Eligibility and controlled re-voting). *Eligibility requires every accepted effective ballot to be associated with a valid election-specific credential issued after authentication of an eligible identity. Controlled re-voting requires that, for each authorized wallet, exactly the accepted ballot with the largest valid sequence number before the deadline contributes to the tally.*

Definition 10 (End-to-end verifiability). *Cast-as-intended means that a voter can challenge ballot creation and verify that the ciphertext represents the intended selection. Recorded-as-cast means that the voter can locate the signed content identifier in the final ledger view and retrieve the identical ballot object. Talled-as-recorded means that any auditor can reconstruct the effective-ballot set, verify all ballot and decryption proofs, recompute the aggregate, and verify the published tally.*

C. Algebraic setting and cryptographic interfaces

A single Type-III BLS12-381 pairing setting is used. Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be cyclic groups of prime order p , with generators $g_1, h_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$, and an efficiently computable pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. The ballot-encryption group is the prime-order BLS12-381 subgroup $\mathbb{G}_E = \mathbb{G}_1$, with a domain-separated generator g_E . The generators h_1 and g_E are derived from fixed, published domain-separation tags with the RFC 9380 hash-to-curve map [33], so that nobody knows a discrete-logarithm relation among g_1 , h_1 , and g_E . This is a security requirement and not a convention: if $\log_{g_1} h_1$ were known, two corrupted identities could open one commitment C to two different attributes, obtain credentials h^{x+ym_1} and h^{x+ym_2} under the same h , compute h^y , and thereby forge credentials for arbitrary wallets. Exponential ElGamal and its Chaum–Pedersen proofs are instantiated in $\mathbb{G}_E$; the credential scheme uses both source groups and the pairing. Under XDH, DDH is assumed hard in $\mathbb{G}_1$, so the ElGamal and credential assumptions are stated over compatible algebraic structures. The target group $\mathbb{G}_T$ is never used for ElGamal, and no composite-order group is introduced.

Definition 11 (DL, DDH, and XDH). *For a prime-order cyclic group $\mathbb{G} = \langle g \rangle$, the discrete-logarithm problem is to recover $x \leftarrow \mathbb{Z}_p$ from g^x . DDH requires the distributions (g, g^x, g^y, g^{xy}) and (g, g^x, g^y, g^z) , for independent uniform x, y, z , to be computationally indistinguishable. XDH requires DDH hardness in $\mathbb{G}_1$ of the Type-III pairing setting.*

Definition 12 (IND-CPA and EUF-CMA). *A public-key encryption scheme is IND-CPA secure if no efficient left-or-right adversary distinguishes encryptions of two equal-length chosen messages with non-negligible advantage. A digital signature is EUF-CMA secure if no efficient adversary, after adaptive signing queries, produces a valid signature on a new message with non-negligible probability.*

1) *Self-contained threshold-issued anonymous credential:* The credential layer is the modified single-private-attribute Coconut/TIAC construction used by the implementation and

analyzed in the cited credential literature [34], [35]. The description below is included so that the protocol and the security reductions do not depend on an unstated primitive.

a) Attribute encoding.: The election-specific credential attribute is

$$m_v = H_{\text{attr}}(\text{enc}(\text{eid}) \parallel \text{enc}(\text{pk}_v) \parallel \text{enc}(T_{\text{cred}})) \in \mathbb{Z}_p,$$

where canonical serialization and the domain-separated hash H_{attr} are fixed in the election parameters. At casting, $(\text{eid}, \text{pk}_v, T_{\text{cred}})$ is public and the verifier recomputes m_v . The hidden issuance attribute is therefore bound to exactly one election wallet.

b) Distributed key generation.: Two degree- $(t_i - 1)$ polynomials $f_x, f_y \in \mathbb{Z}_p[X]$ define $x = f_x(0)$ and $y = f_y(0)$. Issuer I_i receives $(x_i, y_i) = (f_x(i), f_y(i))$ and publishes

$$\text{vk}_i = (\alpha_i, \beta_{2,i}, \beta_{1,i}) = (g_2^{x_i}, g_2^{y_i}, g_1^{y_i}).$$

The aggregate verification key is $\text{mvk} = (\alpha, \beta_2, \beta_1) = (g_2^x, g_2^y, g_1^y)$. A trusted dealer would know (x, y) and could mint credentials alone; PROBA therefore requires a verifiable DKG [36]. Every published key is checked publicly with $e(\beta_{1,i}, g_2) = e(g_1, \beta_{2,i})$, which guarantees that the unblinded shares are well defined.

c) PrepareIssue.: Using a Pedersen-style commitment [37], the voter chooses $o, r \leftarrow \mathbb{Z}_p$ and computes

$$C = g_1^o h_1^{m_v}, \quad h = H_1(C) \in \mathbb{G}_1, \quad D = g_1^r h^{m_v},$$

and produces a Fiat–Shamir proof

$$\pi_{\text{iss}} = \text{PoK}\{(m_v, o, r) : C = g_1^o h_1^{m_v} \wedge D = g_1^r h^{m_v}\}.$$

The proof does not, and need not, show that $m_v = H_{\text{attr}}(\text{eid}, \text{pk}_v, T_{\text{cred}})$: a credential on any other value is useless, because the verifier recomputes m_v from the public wallet key at casting time, and finding a wallet key that hashes to a prescribed value is infeasible. Each issuer separately authenticates the civil identity. The request is $\text{req} = (\text{eid}, C, D, \pi_{\text{iss}})$, while r remains the unblinding state.

d) BlindIssue and Unblind.: After authenticating the civil identity, checking the token, its local record, and π_{iss} , issuer I_i returns

$$\tilde{\sigma}_i = (h, c_i), \quad c_i = h^{x_i} D^{y_i}.$$

The voter unblinds it as

$$s_i = c_i \beta_{1,i}^{-r} = h^{x_i + y_i m_v}, \quad \sigma_i = (h, s_i),$$

and verifies

$$e(h, \alpha_i \beta_{2,i}^{m_v}) = e(s_i, g_2).$$

e) Aggregate.: For a threshold set S of issuer indices, let $\ell_i = \prod_{j \in S, j \neq i} \frac{-j}{i-j} \bmod p$. The voter computes

$$\sigma = (h, s), \quad s = \prod_{i \in S} s_i^{\ell_i} = h^{x + y m_v}.$$

The credential has constant size and can be formed only from a threshold of valid shares under the assumed unforgeability property.

f) Show and Verify.: To show the credential, the voter chooses $\rho \leftarrow \mathbb{Z}_p^*$ and publishes the re-randomized credential

$$\pi_v^{\text{cred}} = (h', s') = (h^\rho, s^\rho).$$

The verifier recomputes m_v from $(\text{eid}, \text{pk}_v, T_{\text{cred}})$, checks $h' \neq 1$, and accepts iff

$$e(h', \alpha \beta_2^{m_v}) = e(s', g_2).$$

Correctness follows because both sides equal $e(h', g_2)^{x + y m_v}$. No proof of knowledge is attached. The attribute is public at showing time, so a proof of knowledge of a blinding exponent, as used for hidden attributes in Coconut, would hide nothing; it would also not bind the ballot, since anyone can derive a fresh proof from a published (h', s') . The binding of the credential to the ballot comes instead from the wallet signature τ_v over cid_v , which covers π_v^{cred} , together with the attribute, which ties the credential to pk_v . Re-randomization hides the issuance signature. Since pk_v is deliberately disclosed, multiple ballots from the same wallet are linkable; the privacy claim is issuance-to-wallet unlinkability, not unlinkability between re-votes.

g) Security properties used by PROBA.: We use four properties. (i) One-more threshold unforgeability (Definition 7): with $f_i < t_i$ corrupted issuers, valid credentials can be obtained only on attributes of requests that received shares from at least $t_i - f_i$ honest issuers, one attribute per request. This property relies on the binding of the commitment C , and hence on the discrete-logarithm assumption with the hash-to-curve generators above, and on the soundness of π_{iss} ; it is the property established for the modified Coconut construction [35]. (ii) Blindness of PrepareIssue/BlindIssue: C and D are perfectly hiding, and π_{iss} is zero knowledge. (iii) Issuance-to-show unlinkability with a disclosed attribute: a showing (h^ρ, s^ρ) is uniformly distributed given the public attribute, independently of the issuance transcript (Theorem 18). (iv) Knowledge soundness and zero knowledge of π_{iss} . The Fiat–Shamir versions are analyzed in the random-oracle model [38], [39].

2) *Remaining cryptographic interfaces:*

- $\mathcal{HE}$ = (KeyGen, Enc, Aggregate, ParDec, Combine): threshold exponential ElGamal in $\mathbb{G}_E$, with IND-CPA security and correct threshold decryption.
- $\mathcal{DS}$ = (KeyGen, Sign, Verify): an EUF-CMA wallet signature. The message contains a protocol version, eid , pk_v , seq_v , and cid_v .
- $\mathcal{ZKP}$: Chaum–Pedersen-style proofs for key possession, ballot validity, the one-selection constraint, and correct partial decryption [40]. Every ballot proof is Fiat–Shamir-bound to $(\text{eid}, \text{pk}_v, \text{seq}_v, \text{ct}_v)$ and to the complete statement (strong Fiat–Shamir), preventing proof replay across wallets or elections; in the random-oracle model this yields the simulation-sound extractability used in Section VI [41], [42].
- H_{cid} : the collision-resistant content-addressing hash used by IPFS. It is distinct from the Fiat–Shamir hash, the attribute hash, and the blockchain transaction hash.

IV. HELIOS AND THE PEREZ–CEESAY BASELINE

A. Helios flow in the same notation

The homomorphic version of Helios is written below using the same setup–casting–tally decomposition as PROBA so that the contribution of PROBA could be clearly pointed.

Algorithm 1: Helios election setup

Input: Security parameter, ballot definition, voter roll, trustee threshold

- 1 The organizer fixes eid , candidates, deadlines, and the voter roll
 - 2 Trustees execute threshold ElGamal key generation and publish mpk with proofs of key possession
 - 3 The organizer publishes and freezes all election parameters on the Helios bulletin board
-

Algorithm 2: Helios ballot preparation and casting

Input: Eligible Helios account, vote vector, mpk

- 1 The browser encrypts the one-out-of- n_c vote and produces ballot-validity and one-selection proofs
 - 2 The voter either audits the ballot, receiving the encryption randomness and discarding that ballot, or elects to cast it
 - 3 For a cast ballot, the voter authenticates to the Helios server
 - 4 The server verifies eligibility and proofs, posts the ciphertext on the bulletin board, and returns a ballot tracker
 - 5 If re-voting is enabled, the election’s published rule determines which authenticated ballot is effective
-

Algorithm 3: Helios effective-ballot selection and tally

Input: Frozen bulletin-board view and trustee shares

- 1 Select the effective authenticated ballot for each voter according to the frozen re-voting rule
 - 2 Verify every ballot proof and homomorphically aggregate the valid ciphertexts
 - 3 Trustees publish partial decryptions and Chaum–Pedersen correctness proofs
 - 4 Combine a threshold of valid shares, publish the result, and allow any auditor to repeat all checks
-

Helios’s E2E-V procedures do not determine how the bulletin board is physically replicated. They also do not provide coercion resistance, receipt freeness, or a software-independent physical record. PROBA preserves Algorithms 1–3 at the ballot and tally layers and changes the authorization and publication layers.

B. Perez–Ceesay blockchain-powered Helios

Perez and Ceesay’s basic flow can be written using the same interfaces as follows.

Algorithm 4: Perez–Ceesay blockchain-powered Helios baseline

Input: Election identifier eid , voter wallet $(\text{sk}_v, \text{pk}_v)$, election key mpk

- 1 **Authorize:** the contract owner authenticates the voter and writes pk_v to an on-chain eligible-wallet set
 - 2 **Prepare ballot:** execute the Helios ballot construction and validity proofs of Algorithm 2
 - 3 **Publish object:** upload $B_v = (\text{eid}, \text{ct}_v, \pi_v^{\text{bal}})$ to IPFS and receive cid_v
 - 4 **Cast:** sign $\tau_v \leftarrow \mathcal{DS}.\text{Sign}(\text{sk}_v, \text{eid} \parallel \text{cid}_v)$ and submit the transaction
 - 5 **Accept:** the contract checks that pk_v is in the eligible set and records the content identifier
 - 6 **Tally:** retrieve accepted objects and execute the Helios proof verification, aggregation, and threshold decryption of Algorithm 3
-

The baseline’s cryptographic ballot and tally are Helios-derived. The differences introduced by PROBA are threshold blind authorization, binding of the proof and signed transaction to the wallet/election/sequence context, an explicit last-valid-ballot rule, and a permissioned bulletin-board deployment.

V. THE PROBA PROTOCOL

A. Architecture and persistent state

Figure 2 gives the high-level architecture. The eligibility registrar and credential issuers are shown separately from the blockchain because a permissioned ledger must not learn the civil identity–wallet association. The smart contract stores election parameters and the latest accepted record for each wallet. IPFS stores the larger encrypted-ballot object; independent pinning providers preserve availability. Because the object is also submitted as transaction calldata, the ledger history contains it as well; IPFS then serves as a content-addressed distribution layer, and audit-data availability follows from either copy.

For each election, the contract maintains

$$\text{Params} = (\text{eid}, \text{mpk}, \text{mvk}, t_i, t_d, T_{\text{reg}}^{\text{open}}, T_{\text{reg}}^{\text{close}}, T_{\text{vote}}^{\text{open}}, T_{\text{vote}}^{\text{close}}, T_{\text{cred}}, S_{\text{max}}, \Delta_{\text{min}}),$$

$$\text{Latest}[\text{pk}] = (\text{seq}, \text{cid}, \tau, t_{\text{last}}).$$

The off-chain registration service maintains an auditable three-state value $\text{RegDB}[\text{eid}, \text{ID}] \in \{\text{unused}, \text{pending}, \text{issued}\}$. It never stores the unblinded wallet key. Before any share is returned, the service atomically changes unused to pending and issues one request identifier bound to the blinded request. All honest issuers consult the same consistency-preserving state, report every returned share to it, and refuse reused request identifiers. After threshold completion, the state becomes issued; an auditable timeout procedure may release a stalled pending registration only if no share has been returned for it. Releasing a request for which honest shares were already returned would be unsafe: corrupted issuers could later complete those shares into a second credential. This

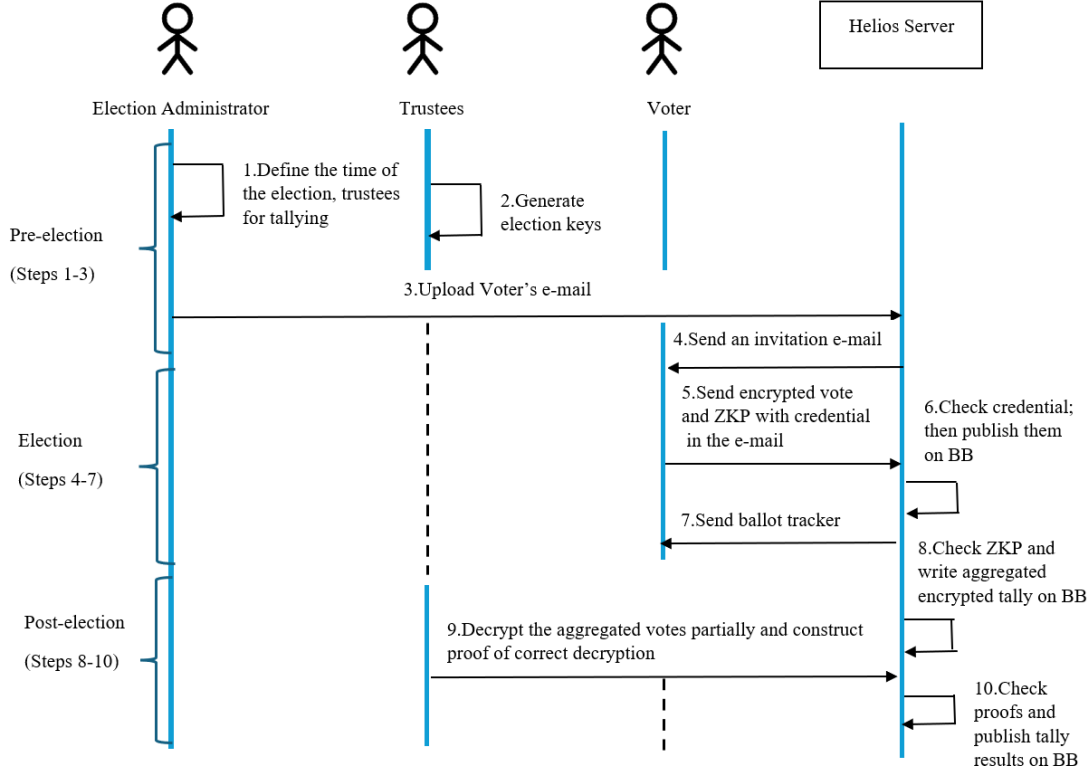


Fig. 1. Helios stage diagram. Algorithms 1–3 are the normative specification used for comparison.

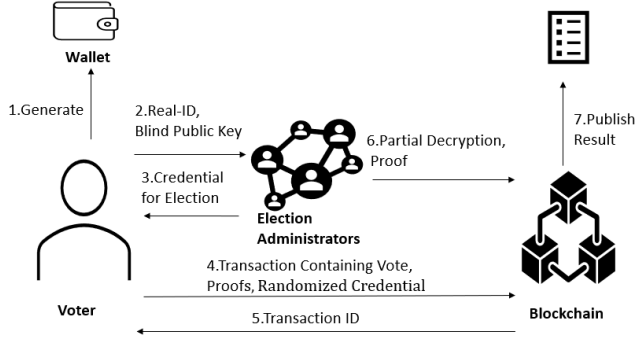


Fig. 2. High-level architecture of PROBA. The original illustration is supplemented by the formal role separation and verification rules in Sections III and V; the blockchain box represents the contract and its IBFT validator consortium, while ballot objects are stored and pinned in IPFS.

atomic reservation prevents concurrent requests from obtaining credentials from disjoint issuer subsets.

The four phases of PROBA are given normatively as Algorithms 5–8.

B. Setup

The setup steps are given in Algorithm 5.

C. Blind registration and credential issuance

The issuance protocol distributes cryptographic authorization but does not remove the need for an authoritative eligibility source in a closed election. Its consistency and denial-of-service behavior are therefore explicit assumptions rather than

Algorithm 5: Election and credential setup

Input: Security parameter 1^λ , election identifier eid , registration/voting windows, credential expiry, thresholds t_i, t_d

- 1 Trustees execute distributed key generation for $\mathcal{HE}$, publish mpk , and prove possession of their shares
- 2 Issuers execute one verifiable $\mathcal{TIAC.DKG}$, obtaining correlated shares isk_i and one aggregate verification key mvk ; every vk_i is checked publicly with $e(\beta_{1,i}, g_2) = e(g_1, \beta_{2,i})$
- 3 The organizer deploys the contract with Params, including the rate-limit parameters $S_{\max}$ and $\Delta_{\min}$, and publishes the ballot definition, proof parameters, hash domains and hash-to-curve tags, deadlines, and archive commitments
- 4 Validators confirm the deployment and independent auditors verify the parameters before the election opens

hidden trust. The improvement is that no single contract owner learns and publishes the identity–wallet list, no single issuer can mint a credential, and an auditable atomic registration state prevents honest disjoint quorums from issuing two live credentials to one identity. Because each issuer authenticates the voter itself, a malicious registration service cannot obtain credentials for identities that it cannot authenticate as, such as abstaining voters; it can only deny service. The local one-share-per-identity record of every honest issuer bounds

Algorithm 6: Blind, threshold wallet authorization

Input: Eligible identity ID_v , election wallet (sk_v, pk_v) , election identifier eid

- 1 The voter computes $(req_v, st_v) \leftarrow \mathcal{TAC}.\text{PrepareIssue}(eid, pk_v, T_{cred})$, where the wallet key is blinded and accompanied by a well-formedness proof
 - 2 The registration service checks that the current time lies in the published pre-registration window, authenticates ID_v , rejects an ineligible or non-unused identity, atomically sets $\text{RegDB}[eid, ID_v] \leftarrow \text{pending}$, and returns a one-use authorization token bound to $H(req_v)$ and $H(ID_v)$
 - 3 Each participating issuer independently authenticates ID_v with a fresh challenge bound to $(eid, i, H(req_v))$, checks that the token names the authenticated identity and that its local record contains no share for ID_v , verifies the one-use authorization token, the common registration-state commitment, and the request proof, then returns $\sigma_{v,i} \leftarrow \mathcal{TAC}.\text{BlindIssue}(isk_i, req_v)$, records ID_v locally, and reports the share to the registration state
 - 4 After receiving at least t_i valid shares, the voter unblinds and aggregates them:
 $\sigma_v \leftarrow \mathcal{TAC}.\text{Aggregate}(\{\mathcal{TAC}.\text{Unblind}(\sigma_{v,i}, st_v)\})$
 - 5 After at least t_i shares have been reported, the registration service sets $\text{RegDB}[eid, ID_v] \leftarrow \text{issued}$ and publishes an auditable completion receipt that contains no wallet key
 - 6 The voter locally verifies σ_v on (eid, pk_v, T_{cred}) and stores the wallet and credential
-

the damage of an inconsistent registration state, and makes identity-level uniqueness independent of the registration service when $2t_i - n_i > f_i$ (Proposition 15).

D. Ballot preparation and casting

Define the ballot core and the complete ballot object by

$$A_v = (eid, pk_v, T_{cred}, seq_v, ct_v, \pi_v^{bal}),$$

$$B_v = (A_v, \pi_v^{cred}), \quad cid_v = H_{CID}(B_v),$$

where $\pi_v^{cred} = (h'_v, s'_v)$ is a fresh re-randomization of σ_v . The wallet signature binds $(eid, pk_v, seq_v, cid_v)$. This ordering avoids a circular hash dependency while binding the wallet, credential, ciphertext, ballot proofs, and stored object. A copied ciphertext or proof cannot be re-submitted under a different wallet because the wallet key occurs in both A_v and the credential attribute, and the complete object, including the credential showing, is authenticated by the wallet signature. The wallet key is intentionally public, providing a stable election pseudonym for controlled re-voting while remaining unlinkable to the civil identity under the anonymous-channel assumption.

The transaction identifier is the hash defined by the blockchain transaction format, not $H(\tau_v)$. Uniqueness follows from the deterministic latest-sequence rule alone; the rate limit of C5 serves availability. With zero-priced transactions and on-chain proof verification, an unlimited re-vote rule would let a single credentialed wallet fill every block at no cost. $S_{\max}$ bounds the total ledger resources of a wallet, and $\Delta_{\min}$ bounds its rate. Transactions that fail C1–C5 still consume block gas if they are included, so validators evaluate Accept before admitting a transaction to the pool. Section VII-C quantifies these bounds.

E. Aggregation and tally

For exponential ElGamal, recovering a small tally from the exponent can use a bounded discrete-log table or another method appropriate for the declared maximum number of voters. The bound and implementation method must be fixed in

the election parameters. How the stated features are achieved is given in Table III.

F. Difference from Helios E2E-V

Cast-as-intended is inherited from the Helios ballot challenge and is not strengthened by the blockchain. Recorded-as-cast changes: the receipt is an actual finalized transaction containing a signed IPFS content identifier, and the ballot is effective only if it is the latest valid sequence for the wallet. Talled-as-recorded changes by making the effective-ballot selection rule public and deterministic before homomorphic aggregation. PROBA additionally provides eligibility verifiability through public credential verification, which is separate from ballot secrecy. The design does not make Helios software-independent, because the voter still needs a correct or successfully challenged client to construct the intended ciphertext.

G. Concrete instantiation

The ballot layer uses exponential ElGamal in the prime-order BLS12-381 subgroup $\mathbb{G}_E = \mathbb{G}_1$ because it is compatible with the Helios homomorphic tally and Chaum–Pedersen proofs. This is a design choice, not a claim that every other homomorphic scheme fails for multiple candidates. Ballot-validity, one-selection, key-possession, and correct-decryption statements are transformed with Fiat–Shamir using domain-separated hashes. The credential layer uses the modified $\mathcal{TAC}$ construction over the BLS12-381 Type-III pairing groups [35], with h_1 , g_E , and H_1 instantiated by RFC 9380 hash-to-curve under fixed domain-separation tags. The wallet signature is the signature algorithm supported by the selected Ethereum-compatible account model. Every encoded statement includes a version identifier and eid to prevent cross-protocol and cross-election replay.

VI. SECURITY ANALYSIS

The analysis is modular and game based. Every experiment is parameterized by 1^λ , uses the corruption limits in

Algorithm 7: Ballot preparation, submission, and acceptance

- Input:** Vote Vote_v , credential σ_v , wallet keys, sequence seq_v , public parameters
- 1 The voter computes $\text{ct}_v \leftarrow \mathcal{HE}.\text{Enc}(\text{mpk}, \text{Vote}_v)$ and $\pi_v^{\text{bal}} \leftarrow \mathcal{ZKP}.\text{ProveBallot}(\text{eid}, \text{pk}_v, \text{seq}_v, \text{ct}_v)$
 - 2 The voter may challenge and discard test encryptions according to the Helios cast-as-intended procedure
 - 3 The voter forms A_v and re-randomizes the credential, $\pi_v^{\text{cred}} \leftarrow \mathcal{TAC}.\text{Show}(\sigma_v)$
 - 4 The voter forms $B_v = (A_v, \pi_v^{\text{cred}})$, uploads it to the configured pinning service, retrieves $\text{cid}_v = H_{\text{cid}}(B_v)$, and verifies the retrieved bytes
 - 5 The voter defines the domain-separated statement $M_v = (\text{PROBA-v3.4}, \text{eid}, \text{pk}_v, \text{seq}_v, \text{cid}_v)$, computes $\tau_v \leftarrow \mathcal{DS}.\text{Sign}(\text{sk}_v, M_v)$, and submits $(B_v, \text{cid}_v, \tau_v)$ as transaction calldata
 - 6 Because an EVM contract cannot fetch IPFS by itself, the contract verifies the supplied ballot object and records only its content identifier and latest-ballot state; the object itself remains in the ledger history as calldata
 - 7 **C1:** check that the current time lies in the voting window, does not exceed T_{cred} , $H_{\text{cid}}(B_v) = \text{cid}_v$, and all embedded identifiers and sequence values match
 - 8 **C2:** check $\mathcal{DS}.\text{Verify}(\text{pk}_v, M_v, \tau_v) = 1$
 - 9 **C3:** check $\mathcal{TAC}.\text{Verify}(\text{mvk}; \text{eid}, \text{pk}_v, T_{\text{cred}}, \pi_v^{\text{cred}}) = 1$
 - 10 **C4:** check $\mathcal{ZKP}.\text{VerifyBallot}(\text{eid}, \text{pk}_v, \text{seq}_v, \text{ct}_v, \pi_v^{\text{bal}}) = 1$
 - 11 **C5:** check that $\text{seq}_v \leq S_{\text{max}}$ and that no current record exists for pk_v , or that $\text{seq}_v > \text{Latest}[\text{pk}_v].\text{seq}$ and the current time is at least $\text{Latest}[\text{pk}_v].t_{\text{last}} + \Delta_{\text{min}}$
 - 12 If every check succeeds, set $\text{Latest}[\text{pk}_v] \leftarrow (\text{seq}_v, \text{cid}_v, \tau_v, \text{now})$, emit an acceptance event, and return the actual blockchain transaction identifier; otherwise reject

Algorithm 8: Effective-ballot reconstruction, aggregation, and tally

- Input:** Final ledger view, IPFS archive, trustee shares
- 1 After the deadline, an auditor reconstructs $S = \{\text{Latest}[\text{pk}] : \text{pk} \text{ has an accepted record}\}$
 - 2 Check that $|S|$ does not exceed the number of published issuance receipts
 - 3 For every record in S , retrieve the exact object addressed by cid from the transaction calldata or from IPFS, recompute H_{cid} , and repeat the signature, credential, and ballot-proof checks from Algorithm 7
 - 4 Reject the tally if an accepted object is missing or any verification fails
 - 5 Compute $\text{ACT} = \mathcal{HE}.\text{Aggregate}(\{\text{ct}_v : v \in S\})$ and publish the effective-ballot list and aggregate
 - 6 Each trustee publishes $\delta_i \leftarrow \mathcal{HE}.\text{ParDec}(\text{dsk}_i, \text{ACT})$ with $\pi_i^{\text{dec}} \leftarrow \mathcal{ZKP}.\text{ProveDec}(\delta_i)$
 - 7 After at least t_d valid proofs, compute $\text{Result} = \mathcal{HE}.\text{Combine}(\text{ACT}, \{\delta_i\})$ and publish it
 - 8 Any auditor repeats all steps and accepts the result only if the recomputed value agrees

TABLE III
MECHANISMS AND RESIDUAL ASSUMPTIONS FOR THE PRINCIPAL PROBA FEATURES.

Feature	Mechanism	Residual assumption / limitation
Threshold authorization	At least t_i valid blind-issuance shares are required, each issuer authenticates the voter independently; the contract verifies the re-randomized credential	A colluding threshold can authorize extra wallets; at least t_i issuers must be online; identity-level uniqueness needs an honest registration state or $2t_i - n_i > f_i$
Registration-to-wallet unlinkability	Blind issuance and re-randomized credential	An anonymity-preserving communication channel is required; same-wallet ballots are linkable by design
Immutable vote storage	Signed content identifier on an append-only finalized ledger; collision-resistant IPFS addressing	Ledger safety and collision resistance; availability needs retained ledger history (calldata) or an honest pinning provider
Privacy-preserving aggregation	Exponential ElGamal, valid-ballot proofs, threshold decryption	Fewer than t_d trustees collude; voter device is not fully compromised
Eligibility verifiability	Public verification of credential proof, wallet signature, election scope, and ballot proof	Correct deployment of the verification code and public parameters
Controlled re-voting	Stable wallet key and monotone sequence; contract keeps the latest valid record, at most S_{max} ballots and one per Δ_{min}	Lost/stolen wallet recovery is not provided
Cost accessibility	Permissioned network may subsidize or configure zero-priced voter transactions	Infrastructure operation still has cost; on-chain verification of C1–C5 costs about 0.89 M gas per ballot for $n_c = 5$ (Section VII-C), which bounds ledger throughput; validators must pre-validate transactions

Assumption 1, and invokes the public deterministic acceptance predicate Accept defined by Algorithm 7. We write FinalView

for the finalized ledger view used by auditors, $f_i < t_i$ for the number of corrupted issuers, and W_H for the set of wallets

TABLE IV
ORACLE INTERFACE USED IN THE SECURITY EXPERIMENTS.

Oracle	Semantics
$\mathcal{O}_{\text{CorruptIssuer}}(i)$	Returns issuer share (x_i, y_i) and the issuer's local record; at most $f_i < t_i$ distinct queries are permitted.
$\mathcal{O}_{\text{CorruptReg}}$	Hands the registration service, including its token key and RegDB, to the adversary.
$\mathcal{O}_{\text{CorruptID}}(\text{ID})$	Marks an eligible identity as corrupted and returns its authentication credential.
$\mathcal{O}_{\text{Register}}(\text{ID})$	For an uncorrupted identity, runs Algorithm 6 honestly with a fresh wallet generated by the challenger, adds the wallet to W_H , and keeps its signing key and credential private.
$\mathcal{O}_{\text{Issue}}(i, \text{ID}, \text{req}, \text{tok})$	For an honest issuer i , authenticates ID, which succeeds for corrupted identities, checks the token, the issuer's local record, the shared state, and π_{iss} , and returns $\mathcal{TZAC}.\text{BlindIssue}(\text{isk}_i, \text{req})$. N_{quor} counts the distinct requests that have received shares from at least $t_i - f_i$ honest issuers.
$\mathcal{O}_{\text{Cast}}(T)$	Applies $\text{Accept}(T)$ to a submitted transaction and updates Latest exactly as Algorithm 7.
$\mathcal{O}_{\text{HonestCast}}(\text{pk}, \text{Vote})$	Prepares and submits a ballot for an honest wallet $\text{pk} \in W_H$.
$\mathcal{O}_{\text{CorruptTrustee}}(i)$	Returns a decryption key share; fewer than t_d distinct trustees may be corrupted in privacy experiments.
$\mathcal{O}_{\text{Finalize}}$	Freezes the ledger, reconstructs the effective-ballot set, publishes valid decryption shares, and returns the public transcript and tally.

generated by honest voters. Because issuance is blind, the challenger cannot associate an issuance session with the wallet that it authorizes. Eligibility is therefore stated in a one-more form that counts issuance requests served by honest issuers rather than wallets.

A. Oracle interface and advantage notation

The experiments use the following stateful oracles. For a bit-guessing experiment, $\text{Adv}(\mathcal{A}) = |\Pr[b' = b] - \frac{1}{2}|$. For a forgery or integrity experiment, the advantage is the probability of the specified winning event.

B. Eligibility and threshold authorization

Security Experiment 13 (Eligibility experiment $\mathcal{G}_{\text{PROBA}, \mathcal{A}}^{\text{elig}}(\lambda)$). The challenger executes setup, gives all public parameters to $\mathcal{A}$, and answers the oracles in Table IV. The adversary may corrupt $f_i < t_i$ issuers, the registration service, and any eligible identities. Let W^* be the set of wallet keys $\text{pk} \notin W_H$ that have an accepted record in FinalView. The adversary wins iff

$$|W^*| > N_{\text{quor}}.$$

Define $\text{Adv}_{\text{PROBA}, \mathcal{A}}^{\text{elig}}(\lambda) = \Pr[\mathcal{G}_{\text{PROBA}, \mathcal{A}}^{\text{elig}} = 1]$.

Each credential requires shares from at least $t_i - f_i$ honest issuers on one request, so N_{quor} is the largest number of non-honest wallets that the honest issuers have legitimately authorized. How many such requests a single identity can obtain is the subject of Proposition 15.

Theorem 14 (Eligibility bound). For every PPT adversary $\mathcal{A}$, there exist PPT reductions $\mathcal{B}_1, \mathcal{B}_2$ such that

$$\text{Adv}_{\text{PROBA}, \mathcal{A}}^{\text{elig}} \leq \text{Adv}_{\mathcal{TZAC}, \mathcal{B}_1}^{\text{omtu}} + \text{Adv}_{H_{\text{attr}}, \mathcal{B}_2}^{\text{coll}}.$$

Hence the eligibility advantage is negligible under one-more threshold unforgeability of the credential and collision resistance of the attribute hash.

Proof. Every record of a wallet in W^* passed C3, so it contains (h', s') with $h' \neq 1$ and $e(h', \alpha\beta_2^m) = e(s', g_2)$ for $m = H_{\text{attr}}(\text{eid}, \text{pk}, T_{\text{cred}})$; this is a valid credential on

m . Distinct wallets yield distinct attributes unless $\mathcal{B}_2$ finds a collision of H_{attr} . $\mathcal{B}_1$ runs the experiment against the one-more threshold-unforgeability challenger of property (i) in Section III-C1. It obtains the corrupted issuers' keys, forwards every honest share requested through $\mathcal{O}_{\text{Issue}}$ or needed by $\mathcal{O}_{\text{Register}}$ to the challenger, and knows the attributes and credentials of the honest wallets in W_H , which it accounts for separately. If $|W^*| > N_{\text{quor}}$, the credentials of W^* are valid credentials on more distinct new attributes than there are requests served by a quorum of honest issuers, which contradicts property (i). The wallet signatures play no role, because the adversary generates the keys of the wallets in W^* itself. $\square$

Proposition 15 (One credentialed wallet per civil identity). Assume that every honest issuer authenticates identities independently, returns at most one share per identity and election, and returns shares only for the request bound to that identity's token. Let $\mathcal{I}^*$ be the set of corrupted identities, and let Adv^{auth} bound the probability that the adversary authenticates as an uncorrupted identity to an honest issuer. Except with probability $\text{Adv}_{\text{PROBA}}^{\text{elig}} + n_i \text{Adv}^{\text{auth}}$,

$$|W^*| \leq \kappa |\mathcal{I}^*|,$$

$$\kappa = \begin{cases} 1, & \text{if the registration service is honest,} \\ & \text{or if } 2t_i - n_i > f_i, \\ \left\lfloor \frac{n_i - f_i}{t_i - f_i} \right\rfloor, & \text{otherwise.} \end{cases}$$

Proof. Under authentication soundness, honest shares for uncorrupted identities are returned only inside $\mathcal{O}_{\text{Register}}$, for wallets in W_H . An honest issuer contributes at most one share per corrupted identity, so each such identity can supply at most $n_i - f_i$ honest shares, and every request counted in N_{quor} consumes at least $t_i - f_i$ of them. An honest registration service reserves a single pending request per identity, and honest issuers serve only that request, so at most one request per identity is counted. If $2t_i - n_i > f_i$, two requests of the same identity would need $2(t_i - f_i) > n_i - f_i$ distinct honest issuers, so again at most one is counted. Theorem 14 completes the argument. $\square$

For the parameters $(n_i, t_i) = (5, 3)$ used in Section VII, the condition $2t_i - n_i > f_i$ holds only for $f_i = 0$; identity-level uniqueness therefore relies on an honest registration service when issuers may be corrupted. The parameters $(n_i, t_i) = (5, 4)$ tolerate $f_i \leq 2$ without that assumption. A malicious registration service cannot, however, obtain credentials for identities that it cannot authenticate as, such as abstaining voters, because each honest issuer authenticates the identity itself. A colluding issuance threshold ($f_i \geq t_i$) can still authorize extra wallets, so trust is distributed rather than eliminated.

C. Controlled re-voting and uniqueness

Theorem 16 (One effective ballot per wallet). *For every wallet public key, the finalized state contains at most one effective ballot, namely the accepted record with the largest sequence number before the deadline.*

Proof. Latest[pk] stores one tuple and changes only after verification of a strictly larger signed sequence number. Equal, stale, cross-election, and cross-wallet statements are rejected, as are sequence numbers above $S_{\max}$ and ballots submitted less than $\Delta_{\min}$ after the previous accepted ballot of the same wallet. Algorithm 8 aggregates exactly the map values in the frozen state. Therefore two effective records for the same wallet cannot occur without a deterministic-contract or ledger-safety failure. $\square$

The property is wallet-level uniqueness; identity-level uniqueness additionally uses Proposition 15. The rate limit does not affect this argument; it bounds the ledger resources that one wallet can consume (Proposition 26).

D. Registration-to-wallet unlinkability

Security Experiment 17 (Registration-to-wallet unlinkability $G_{\text{PROBA}, \mathcal{A}}^{\text{link}}(\lambda)$). *The adversary controls every issuer and the registration service; the issuers' verification keys must pass the public consistency check of Algorithm 5. It chooses two eligible identities ID_0, ID_1 and two wallet keys pk_0, pk_1 . The challenger samples $b \leftarrow \{0, 1\}$ and acts as the two honest voters. It runs Algorithm 6 for (ID_0, pk_b) and (ID_1, pk_{1-b}) with the adversary, in any interleaving that the adversary chooses. If either session fails, both later showings are replaced by $\perp$. Otherwise, the challenger later returns the casting transcripts, including the credential showings, for pk_0 and pk_1 in public-wallet order. The adversary receives no excluded network metadata and may not corrupt either challenge voter. It outputs b' .*

Theorem 18 (Unlinkability bound). *There exists a reduction $\mathcal{B}$ such that*

$$\text{Adv}_{\text{PROBA}, \mathcal{A}}^{\text{link}} \leq 2 \text{Adv}_{\pi_{\text{iss}}, \mathcal{B}}^{\text{zk}}.$$

The bound holds even if every issuer and the registration service collude. It is conditional on the communication-metadata restriction in Assumption 1.

Proof. In hybrid H_1 , the two issuance proofs are simulated, at a cost of at most $2 \text{Adv}_{\pi_{\text{iss}}}^{\text{zk}}$. In H_1 , each issuance transcript

consists of the identity, the token, $C = g_1^o h_1^m$, $h = H_1(C)$, $D = g_1^r h^m$, the simulated proof, and the adversary's own responses. For uniform o and r , C and D are uniformly distributed and independent of m (perfect hiding), so the issuance transcripts are independent of b . A valid share is unique given (h, m) and consistent keys, and the voter rejects every other response, so the responses cannot carry a tag; a rejected share aborts the session, and both showings become $\perp$. Each showing is (h^ρ, s^ρ) with ρ uniform in $\mathbb{Z}_p^*$. Hence h' is uniform in $\mathbb{G}_1 \setminus \{1\}$ and independent of h , and $s' = h'^{x+ym}$ is determined by h' and the public attribute. The casting transcripts therefore depend only on the public wallets and are independent of the issuance sessions, so the guess probability in H_1 is exactly one half. $\square$

The experiment does not model IP addresses, timing, browser fingerprints, or wallet-funding trails. It also does not hide that two accepted records use the same wallet. Unlike the standard unlinkability notion for Coconut-type credentials, the attribute disclosed at showing time is unique to each voter; the argument therefore rests on the perfect hiding of the issuance request and on re-randomization, not on hiding the attribute.

E. Ballot privacy

Security Experiment 19 (Same-tally ballot privacy $G_{\text{PROBA}, \mathcal{A}}^{\text{priv}}(\lambda)$). *The adversary chooses two honest-voter profiles $\mathbf{v}^0, \mathbf{v}^1$ of equal length, with identical public ballot shapes, identical casting schedules including re-votes, and the same aggregate tally. The challenger samples b , constructs all honest ballots from $\mathbf{v}^b$, answers adaptive casts of adversarial ballots and corruptions subject to fewer than t_d corrupted trustees, and returns the finalized public transcript and tally. Challenge-voter secret keys, encryption randomness, and credentials are never revealed. The adversary outputs b' .*

The same-tally restriction removes the unavoidable information published by the result, and identical schedules remove the information that the ledger reveals by design: which wallets voted, when, and how often they re-voted. The experiment is a simplified variant of the BPRIV-style definitions for Helios-like systems [43].

Theorem 20 (Ballot-privacy bound). *Let n_h be the number of honest ballots, including re-votes, each consisting of n_c ciphertexts. There exist reductions such that*

$$\begin{aligned} \text{Adv}_{\text{PROBA}, \mathcal{A}}^{\text{priv}} &\leq n_h n_c \text{Adv}_{\mathcal{HE}}^{\text{ind-cpa}} + \text{Adv}_{\pi_{\text{bal}}}^{\text{zk}} \\ &\quad + \text{Adv}_{\pi_{\text{bal}}}^{\text{ss-ext}} + \text{Adv}_{\pi_{\text{dec}}}^{\text{zk}}. \end{aligned}$$

Proof. H_1 simulates every honest ballot proof by programming the random oracle. H_2 no longer decrypts. It computes the result as the tally of the honest profile $\mathbf{v}^0$ plus the plaintexts extracted from the valid adversarial ballots, and it simulates the honest trustees' decryption shares from the corrupted trustees' shares and this result by Lagrange interpolation in the exponent, which is possible because fewer than t_d trustees are corrupted, together with simulated correctness proofs. H_2 differs from H_1 only if extraction fails in the presence of simulated proofs, which simulation-sound extractability

bounds, or if a simulated decryption proof is distinguished. In H_2 no decryption key is used, so the honest ciphertexts can be switched from $\mathbf{v}^0$ to $\mathbf{v}^1$ one slot at a time with $n_h n_c$ IND-CPA hops. Because the profiles have equal tallies, the computed result is unchanged. An adversarial ballot that copies or transforms an honest ciphertext either fails C4 (Proposition 21) or comes with an extractable witness, and therefore encrypts a plaintext independent of b . Signatures, credential showings, and content identifiers are computed from public values and fresh randomness and add no plaintext oracle. The final hybrid is independent of b . $\square$

Simulation-sound extractability, not only zero knowledge, is required here. With plain soundness, an adversary who sees simulated honest proofs could submit ballots related to honest ones; this is the root of the ballot-copying and weak Fiat–Shamir attacks on Helios [8], [41]. Fiat–Shamir-transformed Σ -protocols whose challenge hashes the complete statement and context, as specified in Section III-C, provide this property in the random-oracle model [41], [42].

F. Ballot independence and copying resistance

Proposition 21 (Context-bound ballot proofs). *Assume simulation-sound extractability of π^{bal} . An adversary who copies $(\text{ct}, \pi^{\text{bal}})$ from wallet pk cannot have the same proof accepted for a different $(\text{eid}', \text{pk}', \text{seq}')$, because the Fiat–Shamir challenge includes the complete context. Producing a new accepted proof for the copied or a related ciphertext requires knowledge of the encryption witness, even after seeing honest proofs. Replacing the content identifier or context under the original wallet additionally requires an EUF-CMA forgery.*

This proposition addresses the ballot-copying issue at the protocol layer. It does not stop a credentialed adversary from independently choosing the same plaintext vote as another voter, which is legitimate behavior.

G. Recorded-as-cast integrity

Security Experiment 22 (Recorded-as-cast integrity $\text{G}_{\text{PROBA}, \mathcal{A}}^{\text{rac}}(\lambda)$). *The challenger gives the adversary an honestly accepted record $R = (\text{eid}, \text{pk}, \text{seq}, \text{cid}, \tau)$ and its object B . The adversary wins by producing $(R^*, B^*) \neq (R, B)$ that an honest voter or auditor accepts as the same finalized record, i.e., the same content identifier resolves to different bytes, a modified signed statement verifies, or the finalized record is replaced without detection.*

Theorem 23 (Recorded-as-cast bound).

$$\text{Adv}_{\text{PROBA}, \mathcal{A}}^{\text{rac}} \leq \text{Adv}_{H_{\text{cid}}}^{\text{coll}} + \text{Adv}_{\text{DS}}^{\text{euf}} + \Pr[\text{LedgerSafetyFail}].$$

Proof. Distinct bytes with one content identifier yield a hash collision. A changed protocol version, election, wallet, sequence, or content identifier with a valid signature yields an EUF-CMA forgery. A different finalized record with neither event violates ledger safety. Because B_v is part of the transaction calldata, every retained copy of the ledger history reproduces the object; the IPFS copy is therefore not needed for integrity, and its unavailability is publicly detectable and is a liveness issue rather than an undetected integrity failure. $\square$

H. Universal and eligibility verifiability

Let $\text{VerifyElection}(\text{FinalView})$ recompute every content identifier, signature, credential showing, ballot proof, effective-ballot choice, aggregate ciphertext, decryption proof, and final tally, and check that the number of effective wallets does not exceed the number of published issuance receipts.

Security Experiment 24 (Universal-verifiability experiment $\text{G}_{\text{PROBA}, \mathcal{A}}^{\text{uv}}(\lambda)$). *The adversary controls arbitrary voters, fewer than the stated issuer/trustee thresholds, and the public communication schedule within the ledger assumptions. It outputs a finalized election view. It wins iff VerifyElection accepts but the published result differs from the deterministic tally of (i) the final valid ballots of honest voters that are recorded in FinalView and (ii) any valid ballots cast by corrupted eligible voters, under the latest-sequence rule.*

Theorem 25 (Universal- and eligibility-verifiability bound). *For suitable reductions,*

$$\begin{aligned} \text{Adv}_{\text{PROBA}, \mathcal{A}}^{\text{uv}} \leq & \text{Adv}_{\text{PROBA}}^{\text{elig}} + \text{Adv}_{\pi^{\text{bal}}}^{\text{snd}} + \text{Adv}_{\pi^{\text{dec}}}^{\text{snd}} \\ & + \text{Adv}_{H_{\text{cid}}}^{\text{coll}} + \text{Adv}_{\text{DS}}^{\text{euf}} + \Pr[\text{LedgerSafetyFail}] \\ & + \Pr[\text{HECorrectnessFail}]. \end{aligned}$$

Proof. An accepted ineligible ballot invokes the eligibility event. An included malformed selection or false decryption share violates the corresponding proof soundness. Substituting an object or transaction invokes collision resistance or signature unforgeability. Omitting or rewriting a finalized accepted record violates ledger safety. If none of these events occurs, the public verifier reconstructs the unique latest valid record for every wallet, aggregates exactly those ciphertexts, and combines only verified trustee shares. A different accepted result then contradicts homomorphic and threshold-decryption correctness. $\square$

The inclusion of an honest voter’s ballot is not a universal-verifiability property: a Byzantine proposer may delay a transaction, and near the deadline the transaction may be excluded. The voter detects this through the recorded-as-cast check, and ledger liveness under the IBFT fault bound determines whether a resubmission is included before $T_{\text{vote}}^{\text{close}}$. Cast-as-intended remains the Helios challenge procedure: its assurance depends on the voter auditing one or more independently generated test ballots. The blockchain does not repair a voter client that is never challenged or independently checked.

I. Conditional robustness and availability

Proposition 26 (Conditional safety and liveness). *Under Assumption 1: (i) credential safety holds with fewer than t_i corrupted issuers; (ii) registration liveness requires at least t_i responsive issuers and a responsive registration service; (iii) ballot secrecy holds with fewer than t_d corrupted trustees; (iv) tally liveness requires at least t_d responsive trustees; (v) ledger safety and liveness require the configured IBFT fault bound; (vi) audit-data availability requires that at least one honest validator or archive retains the ledger history, which contains every accepted B_v , or that at least one honest pinning provider retains each accepted object; and (vii) ledger capacity during*

the voting window requires validators to evaluate *Accept* before admitting a transaction to the pool, since rejected transactions that are included still consume block gas at zero price, and each wallet can consume at most $S_{\max}$ accepted ballots, at most one per $\Delta_{\min}$.

Thus robustness is a composition of threshold safety and conditional liveness, not an unconditional claim that blockchain eliminates trust. The experiments, oracle restrictions, algebraic settings, and advantage bounds in this section constitute the security specification used by the protocol.

VII. IMPLEMENTATION AND PERFORMANCE EVALUATION

This section replaces the legacy microbenchmarks of the previous submission, which measured a 4096-bit finite-field ElGamal instantiation that is no longer part of the protocol. All measurements below were newly collected for this revision on the BLS12-381 instantiation of Section III-C. They exercise the normative Algorithms 5–8, including independent issuer authentication and the rate-limited acceptance rule. The benchmark suite, the smart contracts, the raw timing samples, and the scripts that regenerate every table, figure, and number in this section are released with the implementation.

A. Implementation and measurement setup

a) Reference implementation.: The protocol logic is written in Python. Group and pairing operations on BLS12-381, RFC 9380 hash-to-curve for H_1 , h_1 , and g_E , and multi-scalar multiplication are executed by the Rust arkworks backend through `py_arkworks_bls12381` 0.5.0. The wallet uses `secp256k1` through `coincurve` 21.0.0 (`libsecp256k1`). Issuer-side identity authentication is represented by an Ed25519 challenge–response against an authentication key that every issuer enrolls through its own channel; a deployment substitutes its eID or single-sign-on mechanism. The implementation follows Section III-C1 literally: the attribute $m_v = H_{\text{attr}}(\text{eid} \parallel \text{pk}_v \parallel T_{\text{cred}})$, the issuance proof π_{iss} , unblinding with the share-verification pairing, Lagrange aggregation, and the re-randomized showing $\pi_v^{\text{cred}} = (h'_v, s'_v)$. Ballots use one exponential-ElGamal ciphertext per candidate in $\mathbb{G}_E = \mathbb{G}_1$, a disjunctive Chaum–Pedersen proof per slot, and one one-selection proof. Every Fiat–Shamir challenge contains $(\text{eid}, \text{pk}_v, \text{seq}_v, \text{keccak}(\text{ct}_v))$ and the complete statement, and is computed as a `keccak256` output reduced from 512 bits modulo p , so that exactly the same function is evaluated off-chain and in the contract. The wallet statement M_v is signed with an EIP-191 `secp256k1` signature and verified with `ecrecover`.

b) Canonical encoding and content identifiers.: The ballot object $B_v = (A_v, \pi_v^{\text{cred}})$ is serialized in the EIP-2537 wire format (128 bytes per $\mathbb{G}_1$ point and 32 bytes per scalar). Hence the same bytes are hashed into cid_v , pinned in IPFS, submitted as `calldata`, and consumed by the contract without re-encoding. H_{cid} is CIDv1 with the raw codec and SHA2-256. For single-block objects (at most 256 KiB, which covers every ballot considered here), we verified that it produces byte-identical identifiers to `ipfs add -cid-version=1`

`-raw-leaves` of kubo 0.38.1. For n_c candidates, $|B_v| = 512 + 384n_c$ bytes (2,432 B for $n_c = 5$); with compressed points the same content would occupy 1,472 B, but it would then require on-chain point decompression.

c) Smart contracts.: `ProbaElection` (8,015 bytes of runtime code) implements checks C1–C5 and the state $\text{Latest}[\text{pk}] = (\text{seq}, \text{cid}, \tau, t_{\text{last}})$ of Algorithm 7. C3 and C4 use the BLS12-381 precompiles of EIP-2537. Every point decoded from the ballot object enters a multi-scalar-multiplication or pairing precompile, which performs the subgroup check; $\mathbb{G}_1$ addition, which performs no subgroup check, is applied only to already-checked points. The election parameters $(g_E, \text{mpk}, g_2^{-1}, \alpha, \beta_2)$ are stored as the code of a data contract (SSTORE2) and read with one `EXTCODECOPY` per value. The Perez–Ceesay baseline of Algorithm 4 is implemented as `PerezCeesayElection`, in which the owner writes eligible addresses and a cast records the content identifier and the signature. Contracts are compiled with `solc` 0.8.28 (`-via-ir`, optimizer with 200 runs) and executed on Foundry `anvil` 1.5.1 with the Prague hardfork, zero base fee, and zero gas price. Every voter submits from its own zero-balance election wallet, which realizes the subsidized-fee policy of Definition 4 without any faucet or token transfer. Unless stated otherwise, $S_{\max} = 16$ and $\Delta_{\min} = 60$ s.

d) Platform and statistics.: All reported experiments ran on an Apple M4 (4 performance + 6 efficiency cores) with 24.0 GiB memory under macOS 26.6.2 with Python 3.13.1. The protocol roles execute sequentially in one single-threaded Python process, while `anvil` and the IPFS node run as separate local processes. Each primitive was timed after five warm-up runs with garbage collection disabled, using 500 repetitions for operations below 0.1 ms, 200 repetitions for protocol operations, and 30 repetitions for the distributed key generations. We report the mean, standard deviation, median, and a Student- t 95% confidence interval (CI) of the mean. For the parameter sweeps, all configurations were sampled in an interleaved round-robin order so that transient system load affects every configuration equally. Unless stated otherwise, $n_i = 5$, $t_i = 3$, $n_d = 5$, $t_d = 3$, and $n_c = 5$. Gas values are deterministic up to `calldata` zero-byte variation and are averaged over eight voters per configuration. An independent repetition of the complete suite on an Intel(R) Xeon(R) Processor @ 2.80GHz (1 logical CPU, Ubuntu 24.04.4 LTS) reproduced every gas value to within 72 gas and all differential-test verdicts, with CPU times 2.0 to 3.1 times longer (median 2.6) for operations above 0.1 ms.

B. Primitive and per-role costs

Table V reports every CPU operation that occurs in Algorithms 5–8. The credential operations that PROBA adds to Helios are small. Showing a credential is a re-randomization that takes $T_{\text{PrCred}} = 0.20$ ms, and verifying it takes $T_{\text{VrCred}} = 0.93$ ms, dominated by one $\mathbb{G}_2$ scalar multiplication and a two-pair pairing check; the credential-specific work across the voter and one verifier is therefore 1.13 ms. The Helios-derived ballot layer dominates. For $n_c = 5$, the voter’s complete ballot preparation, which comprises encryption, the per-slot and one-selection proofs, the credential showing, the content identifier,

TABLE V
PRIMITIVE-LEVEL CPU COST OF PROBA ON BLS12-381 (MS). ELGAMAL, PROOF, AND DECRYPTION TIMINGS ARE PER CANDIDATE SLOT UNLESS STATED OTHERWISE; $n_i = 5$, $t_i = 3$, $n_d = 5$, $t_d = 3$.

Notation	Operation	Mean	SD	Median	95% CI
<i>TIAC</i>					
T_{Hattr}	Attribute hash $m_v = H_{attr}(eid, pk, T_{cred})$	0.006	0.001	0.006	[0.006, 0.006]
T_{Prep}	PrepareIssue incl. π_{iss}	0.878	0.019	0.876	[0.876, 0.881]
T_{ReqVer}	Verify π_{iss} (issuer side, incl. H_1)	0.528	0.014	0.530	[0.526, 0.530]
T_{Issue}	BlindIssue incl. π_{iss} verification	0.686	0.016	0.687	[0.684, 0.688]
$T_{Unblind}$	Unblind one share incl. pairing check	0.987	0.014	0.987	[0.985, 0.989]
$T_{AggCred}$	Aggregate $t_i=3$ shares incl. verification	1.157	0.019	1.156	[1.154, 1.160]
T_{PrCred}	Show credential (re-randomisation)	0.199	0.009	0.200	[0.199, 0.200]
T_{VrCred}	Verify credential showing (C3)	0.927	0.021	0.921	[0.924, 0.930]
<i>Registration</i>					
T_{Auth}	Registrar: window/eligibility, RegDB unused→pending, token	0.026	0.003	0.025	[0.025, 0.026]
T_{IdResp}	Voter: identity-authentication response to one issuer	0.014	0.001	0.014	[0.014, 0.014]
T_{IssChk}	Issuer: identity authentication, token, record and state checks	0.083	0.004	0.083	[0.082, 0.083]
<i>ElGamal</i>					
T_{Enc}	Encrypt one candidate slot	0.235	0.048	0.225	[0.230, 0.239]
T_{ParDec}	Partial decryption of one slot	0.086	0.004	0.085	[0.086, 0.087]
T_{Comb}	Combine $t_d=3$ shares + BSGS (bound 10^4)	0.255	0.010	0.256	[0.254, 0.257]
T_{Chal}	Benaloh audit of one slot (re-encryption)	0.206	0.006	0.207	[0.205, 0.207]
<i>Chaum-Pedersen</i>					
T_{PrVV}	Prove 0/1 validity of one slot	0.582	0.016	0.583	[0.580, 0.584]
T_{VrVV}	Verify 0/1 validity of one slot	0.707	0.010	0.708	[0.706, 0.709]
T_{PrVS}	Prove one-selection constraint	0.223	0.011	0.223	[0.221, 0.224]
T_{VrVS}	Verify one-selection constraint	0.366	0.010	0.368	[0.364, 0.367]
T_{PrCD}	Prove correct partial decryption	0.227	0.010	0.227	[0.225, 0.228]
T_{VrCD}	Verify correct partial decryption	0.346	0.011	0.349	[0.344, 0.347]
<i>Wallet/encoding</i>					
T_{Sign}	Wallet signature τ_v (secp256k1, EIP-191)	0.017	0.001	0.017	[0.016, 0.017]
T_{SigVr}	Wallet signature verification (ecrecover)	0.021	0.001	0.021	[0.021, 0.021]
T_{CID}	Content identifier $H_{cid}(B_v)$ ($n_c=5$)	0.001	0.000	0.001	[0.001, 0.001]
T_{Parse}	Decode B_v incl. subgroup checks ($n_c=5$)	0.589	0.017	0.594	[0.587, 0.591]
<i>Composite</i>					
T_{Voter}	Complete ballot preparation (Alg. 7, steps 1,3-5), $n_c=5$	4.827	0.057	4.835	[4.820, 4.835]
T_{Accept}	Complete off-chain Accept C1–C5 (Python), $n_c=5$	5.721	0.073	5.713	[5.711, 5.731]
<i>Setup</i>					
T_{DKG-HE}	Trustee DKG + key-possession proofs ($n_d=5$, $t_d=3$)	5.536	0.071	5.550	[5.509, 5.562]
$T_{DKG-TIAC}$	Issuer DKG with Feldman checks ($n_i=5$, $t_i=3$)	30.404	0.384	30.400	[30.261, 30.548]

and the wallet signature, takes 4.83 ms on average. The complete off-chain evaluation of the acceptance predicate C1–C5, including decoding with subgroup checks, takes 5.72 ms. The composites exceed the sums of their cryptographic components ($n_c(T_{Enc} + T_{PrVV}) + T_{PrVS} + T_{PrCred} = 4.51$ ms and $T_{Parse} + n_c T_{VrVV} + T_{VrVS} + T_{VrCred} + T_{SigVr} = 5.44$ ms) by steps that the component timings do not include: serialization and hashing of the complete ciphertext vector for the proof context, the homomorphic sums for the one-selection proof, and object assembly.

Table VI maps the measurements to the four protocol phases and states what remains outside each figure.

a) Scaling.: Figure 3 and Tables VII and VIII show that every per-role cost is linear in the relevant parameter. Ballot preparation grows by approximately 0.89 ms and verification by approximately 0.90 ms per additional candidate. Registration grows by approximately 2.1 ms per additional issuer when all roles run in one process. Of this increase, about 1.3 ms falls on the voter, who answers one authentication challenge and verifies each share with a pairing, and about 0.8 ms falls on the issuers, which can respond in parallel. The per-trustee

decryption cost is independent of t_d , whereas the auditor’s verification of the partial-decryption proofs grows linearly with t_d .

C. On-chain cost of the acceptance predicate

Table IX decomposes the gas of one accepted ballot by the checks of Algorithm 7; Figure 4 shows the same data. Across the measured range, the cost follows $\approx 319,700 + 113,800 n_c$ gas. C3 costs 134.0–134.3 k gas, nearly independent of n_c : one single-term $\mathbb{G}_2$ multi-scalar multiplication, one $\mathbb{G}_2$ addition, and a two-pair pairing check. C4 costs approximately 108 k gas per candidate, mainly for the four $\mathbb{G}_1$ multi-scalar multiplications that recompute the commitments of each disjunctive proof. For $n_c = 5$, C3 and C4 represent 82.6% of the 888 k gas of a first ballot. C1 and C2 (layout, context, time window, content identifier, and `ecrecover`) cost only 6.3 k gas. The first write of `Latest[pk]` with its event costs 92.4 k gas; the rate-limit timestamp t_{last} shares a storage slot with `seq` and adds no storage write. A re-vote, which overwrites the same slots, costs 68.3 k gas less than a first ballot.

TABLE VI
MEASURED COST PER PROTOCOL PHASE AND ROLE FOR $n_i = 5$, $t_i = 3$, $n_d = 5$, $t_d = 3$, $n_c = 5$. CPU VALUES ARE MEDIAN OF THE INTERLEAVED SWEEPS OR MEANS OF TABLE V; GAS IS MEASURED ON THE EIP-2537 EVM.

Protocol step	Measured cost	Not included
Setup, Algorithm 5	Trustee DKG with key-possession proofs: 5.54 ms in total; issuer DKG with Feldman checks: 30.40 ms in total; contract deployment including the parameter blob: 2.05 million gas	Network rounds and complaint handling of the DKGs
Blind registration, Algorithm 6	Voter CPU 5.91 ms, including t_i authentication responses; per issuer $T_{\text{IssChk}} + T_{\text{Issue}} = 0.77$ ms; registration service 0.026 ms; all roles executed sequentially 8.47 ms; no ledger transaction	Real civil-identity authentication, anonymous-channel and inter-party network latency
Ballot preparation, Algorithm 7 (voter)	4.83 ms CPU; IPFS add and pin 16.20–17.71 ms and retrieve-and-compare 0.40–0.66 ms on a local node	Benaloh-challenge repetitions ($T_{\text{Chal}} = 0.21$ ms per slot and repetition), browser runtime
Contract acceptance, checks C1–C5	887,717 gas for a first ballot and 819,409 gas for a re-vote; indicative EVM execution time 3.6 ms per ballot (difference of <code>eth_call</code> latencies)	IBFT consensus and finality, validator networking
Audit and tally, Algorithm 8	Per effective ballot: 5.72 ms off-chain re-verification plus 0.92 ms retrieval from the ledger or 0.56 ms from IPFS; per trustee: 1.68 ms for n_c partial decryptions with proofs; verification of t_d shares and combination: 7.16 ms	Wide-area retrieval

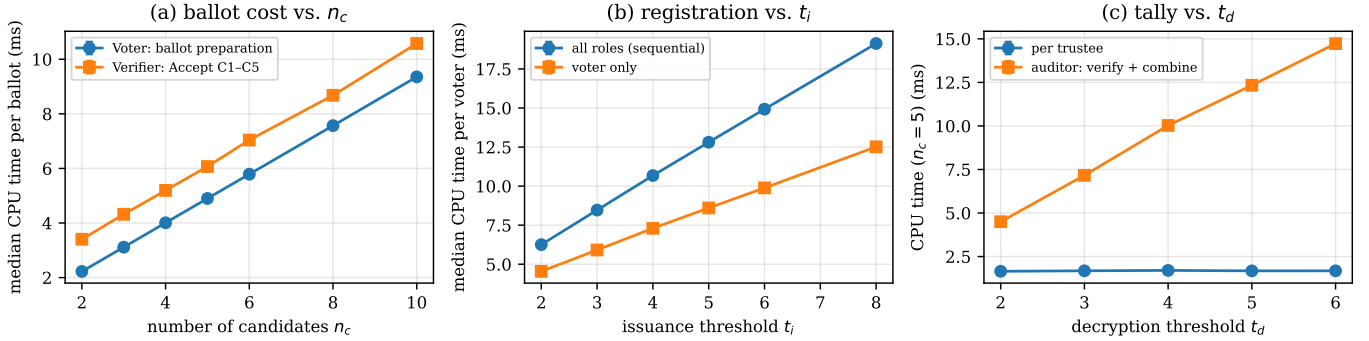


Fig. 3. Scaling of per-role CPU time (median; error bars show the 95% CI of the mean). (a) Voter ballot preparation and off-chain evaluation of C1–C5 versus n_c . (b) Registration of one voter versus the issuance threshold t_i , for all roles executed sequentially and for the voter alone. (c) Tally work versus the decryption threshold t_d for $n_c = 5$.

TABLE VII
OBJECT SIZE, MEDIAN CPU TIME, AND GAS AS FUNCTIONS OF THE NUMBER OF CANDIDATES n_c . “WIRE” IS THE EIP-2537 ENCODING USED FOR `cid_v` AND `CALLDATA`; “COMPR.” IS THE SAME CONTENT WITH COMPRESSED POINTS.

n_c	$ B_v $ wire (B)	$ B_v $ compr. (B)	Voter (ms)	Accept, Python (ms)	First-vote gas	Re-vote gas
2	1,280	800	2.22	3.41	547,197	478,908
3	1,664	1,024	3.11	4.32	660,604	592,307
4	2,048	1,248	4.01	5.19	774,096	705,760
5	2,432	1,472	4.90	6.07	887,717	819,409
6	2,816	1,696	5.79	7.04	1,001,423	933,096
8	3,584	2,144	7.57	8.68	1,229,141	1,160,869
10	4,352	2,592	9.36	10.58	1,457,291	1,389,006

TABLE VIII
REGISTRATION COST OF ONE VOTER (ALGORITHM 6) VERSUS THE ISSUANCE THRESHOLD; MEDIAN IN MS. “BLINDISSUE” INCLUDES EACH ISSUER’S IDENTITY AUTHENTICATION, TOKEN, RECORD, AND STATE CHECKS; ISSUER WORK IS EXECUTED SEQUENTIALLY IN THE SAME PROCESS.

(n_i, t_i)	PrepareIssue	$t_i \times \text{BlindIssue}$	$t_i \times \text{Unblind}$	Aggregate	Total	Voter only
(3,2)	0.89	1.68	2.38	1.22	6.25	4.54
(5,3)	0.89	2.52	3.59	1.37	8.47	5.91
(7,4)	0.90	3.34	4.80	1.54	10.68	7.30
(9,5)	0.90	4.18	6.00	1.62	12.81	8.60
(11,6)	0.90	5.01	7.16	1.72	14.93	9.89
(15,8)	0.90	6.60	9.57	1.91	19.13	12.52

TABLE IX
GAS PER ACCEPTED BALLOT BY CHECK OF ALGORITHM 7. C1–C4 ARE MEASURED IN-CONTRACT WITH `GASLEFT()`; “C5+STORE” IS THE FIRST-BALLOT RESIDUAL, INCLUDING THE NEW STORAGE SLOTS OF `Latest[pk]` AND THE ACCEPTANCE EVENT; EVERY ROW ADDITIONALLY CONTAINS THE 21,000 INTRINSIC GAS.

n_c	Calldata	C1+C2	C3	C4	C5+store	First vote	Re-vote
2	19,020	5,650	134,303	274,803	92,421	547,197	478,908
3	24,394	5,869	134,264	382,651	92,426	660,604	592,307
4	29,750	6,088	134,225	490,603	92,431	774,096	705,760
5	35,127	6,308	134,185	598,661	92,436	887,717	819,409
6	40,485	6,529	134,144	706,824	92,441	1,001,423	933,096
8	51,192	6,972	134,061	923,464	92,452	1,229,141	1,160,869
10	61,911	7,416	133,987	1,140,515	92,462	1,457,291	1,389,006

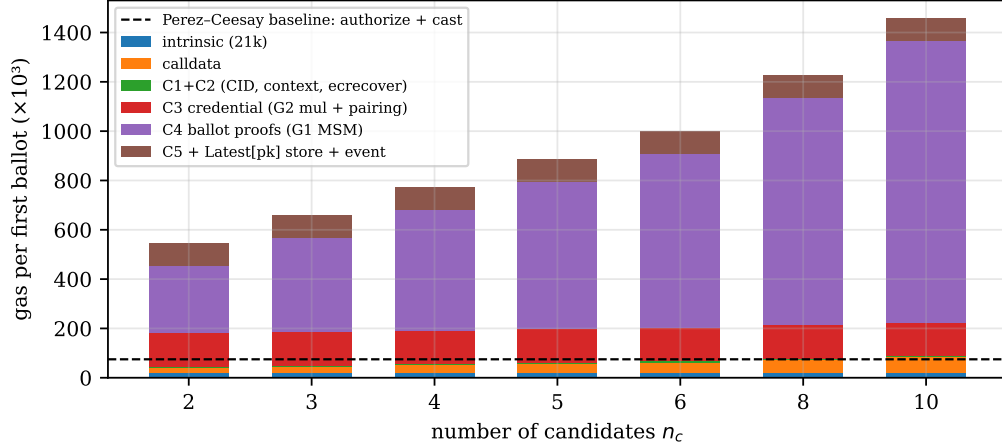


Fig. 4. Gas composition of a first accepted ballot versus n_c , compared with the per-voter on-chain cost of the Perez–Ceesay baseline (batched owner authorization plus cast).

a) Comparison with the Perez–Ceesay baseline.: In the baseline, the owner’s authorization costs 24,264 gas per wallet when 100 wallets are written in one transaction (45,754 gas for a single-wallet transaction), and a cast costs 50,545 gas (33,450 gas for an overwrite). The per-voter on-chain cost of the baseline is therefore about 74.8 k gas, compared with 888 k gas for PROBA at $n_c = 5$. PROBA removes the owner-maintained eligibility list, and with it the per-wallet authorization transaction, but it is approximately 12 times more expensive in gas. The difference is not caused by the authorization mechanism itself: it arises because Algorithm 7 verifies the credential and ballot proofs on-chain, whereas the baseline defers all proof verification to the tally. Without C3 and C4, the PROBA transaction would cost approximately 155 k gas. We keep on-chain verification because it makes the ledger contain only well-formed, credentialed records and makes the acceptance decision itself publicly verifiable. A deployment could instead defer C3 and C4 to auditors, since Algorithm 8 already repeats them, but it would then have to specify how ill-formed records are excluded at tally time.

b) Implications for the permissioned deployment.: With a zero gas price, gas is not a voter-facing cost; it bounds the ledger’s capacity. At $n_c = 5$, a block gas limit of 30, 60, or 100 million admits 33, 67, or 112 ballots per block, i.e. at most 16.5, 33.5, or 56 ballots per second with a 2-second IBFT block period. For example, 10,000 first ballots would

require 304 blocks, or about 10 minutes of ledger capacity, at a 30-million gas limit. These values are analytic upper bounds derived from the measured gas; they are not measured IBFT throughput. The same numbers quantify the denial-of-service risk addressed by the rate limit of C5. Without it, a single credentialed wallet could submit 36 re-votes per 30-million-gas block at no cost. With $S_{\max} = 16$ and $\Delta_{\min} = 60$ s, a wallet can consume at most 13.2 million gas over the whole election, and at least 990 distinct credentialed wallets acting together would be needed to keep the ledger saturated. Transactions that fail C1–C5 still consume block gas when they are included; validators must therefore evaluate `Accept` before admitting a transaction to the pool, which costs 5.72 ms of CPU per candidate transaction in our implementation. The contract deployment, including the parameter data contract, costs 2.05 million gas once per election, compared with 0.27 million gas for the baseline contract. The per-candidate cost of C4 could be reduced by batch verification of the Chaum–Pedersen equations, which would require including the proof commitments in π_v^{bal} ; we leave this optimization to future work.

D. Functional and adversarial validation

To check that the contract enforces the same predicate as the specification, we submitted valid and deliberately malformed

TABLE X
DIFFERENTIAL VALIDATION OF THE ACCEPTANCE PREDICATE: REVERT REASON OF THE CONTRACT AND VERDICT OF THE PYTHON IMPLEMENTATION
($n_c = 5$).

Submitted transaction	Contract	Python Accept	Agree
Valid first ballot	accepted	accepted	yes
Exact replay of an accepted transaction	C5:seq	C5:sequence	yes
Re-vote before the minimum interval $\Delta_{\min}$	C5:interval	C5:interval	yes
Re-vote with larger sequence number after $\Delta_{\min}$	accepted	accepted	yes
Sequence number above $S_{\max}$	C5:seq	C5:sequence	yes
Stale re-vote (seq_v not larger than Latest)	C5:seq	C5:sequence	yes
Modified object bytes under the original cid_v/τ_v	C1:cid	C1:cid	yes
Signature by a different wallet	C2:sig	C2:signature	yes
Uncredentialed wallet with a forged credential	C3:cred	C3:credential	yes
Credential transferred to another wallet	C3:cred	C3:credential	yes
Ballot copying: victim's ciphertext and proof under own credential	C4:ballot	C4:ballot-proof	yes
Proof replay to another sequence number	C4:ballot	C4:ballot-proof	yes
Over-vote (two selections)	C4:ballot	C4:ballot-proof	yes
Non-binary slot (weight 2)	C4:ballot	C4:ballot-proof	yes
Empty ballot (no selection)	C4:ballot	C4:ballot-proof	yes
Ciphertext point outside the prime-order subgroup	precompile	C1:parse	yes
Cross-election replay (ballot of election 1 on election 2)	C1:eid	C1:context	yes
Expired credential (block time $> T_{\text{cred}}$)	C1:time	C1:time	yes
Object for a different ballot shape ($n_c = 4$)	C1:len	C1:context	yes

transactions to both the contract and an independent Python implementation of Accept (Table X). All 19 cases produced the expected decision, and the two implementations agreed in every case. In the table, a rejection counts as agreement when both implementations reject; they may report different checks because the Python decoder validates subgroup membership and layout before C3, whereas the contract detects the same faults in C1 or inside a precompile. The test suite additionally checks the registration layer: a registration service that issues a token for an abstaining identity obtains no share, because it cannot answer the issuers' authentication challenges; an honest issuer refuses a second share for the same identity even when the registration state is reset; a pending registration can be released only while no share has been returned; and, if the discrete logarithm of h_1 were known, two registrations would suffice to forge credentials for arbitrary wallets (Section III-C1). These tests are not a substitute for the proofs in Section VI; they show that the implementation rejects the attack patterns considered there, including ballot copying (Proposition 21), proof replay across sequence numbers and elections, credential transfer between wallets, stale and premature re-votes (Theorem 16).

E. End-to-end election runs

Finally, we executed complete elections with 100, 250, 500, and 1,000 voters and $n_c = 5$. Each voter generated an election wallet and registered blindly with a randomly chosen quorum of $t_i = 3$ of the $n_i = 5$ issuers, authenticating itself to each of them. It then prepared a ballot, pinned it in a local IPFS node, retrieved and compared the stored bytes, and submitted the transaction from its own wallet. After all first ballots and a pause of $\Delta_{\min}$, 20% of the voters, chosen at random, re-voted with $\text{seq}_v = 2$. The auditor then reconstructed Latest[.] from the acceptance events, cross-checked it against the contract state, and checked that the number of effective wallets did not exceed the number of issuance receipts. It recovered every effective object from the transaction calldata and, independently,

from IPFS by its content identifier, and checked that both copies were identical. It then repeated C1–C5, aggregated the ciphertexts, verified $t_d = 3$ partial decryptions, and recovered the tally by a baby-step giant-step search bounded by n_v . In every run, the tally matched the plaintext ground truth and counted only the latest ballot of each re-voting wallet (Table XI and Figure 5).

All phases scale linearly in n_v . Registration costs 8.57–10.37 ms per voter, and audit and tally cost 11.7 s for 1,000 effective ballots, of which the threshold decryption and combination take 0.01 s. Recovering an object from the ledger calldata takes 0.92 ms, so the audit does not depend on the IPFS copy. A cast takes 30–37 ms end to end. Its largest component is adding and pinning the object in the local IPFS node, not a cryptographic operation. Ballot preparation takes 5.28–5.68 ms, adding and pinning the object in the local IPFS node, which includes writing it to the node's on-disk datastore, takes 16.20–17.71 ms, and transaction submission with receipt takes 7.93–12.95 ms. The submission time increases as the development chain grows to 1,200 single-transaction blocks; this is a property of the development node and should not be read as ledger latency.

F. Threats to validity and scope of the claims

The measurements support conclusions about computational and ledger-resource cost, not about deployed latency. (i) All roles ran sequentially in one process on a single machine; issuers, trustees, and validators would run in parallel on separate machines, and native or WebAssembly clients may differ from the Python orchestration layer, although the group arithmetic is executed in native code. The operating system may schedule the process on cores of different performance classes. The observed dispersion was nevertheless low: the median coefficient of variation over the operations above 0.1 ms is 2.6%, and the largest is 20% for T_{Enc} (median 0.23 ms). (ii) anvil executes the EVM and prices gas exactly for the

TABLE XI
COMPLETE ELECTION RUNS WITH $n_c = 5$, $t_i = 3$ OF $n_i = 5$, $t_d = 3$ OF $n_d = 5$, AND 20% RE-VOTING. TIMES ARE WALL-CLOCK, WITH ALL ROLES EXECUTED SEQUENTIALLY.

n_v	Tx	Setup (s)	Registration (s)	Casting (s)	Audit+tally (s)	Gas/tx	Tally correct
100	120	0.05	0.87	3.64	0.95	876,307	yes
250	300	0.04	2.15	9.02	2.35	876,322	yes
500	600	0.05	4.56	20.38	5.83	876,321	yes
1,000	1,200	0.06	10.41	44.41	11.75	876,320	yes

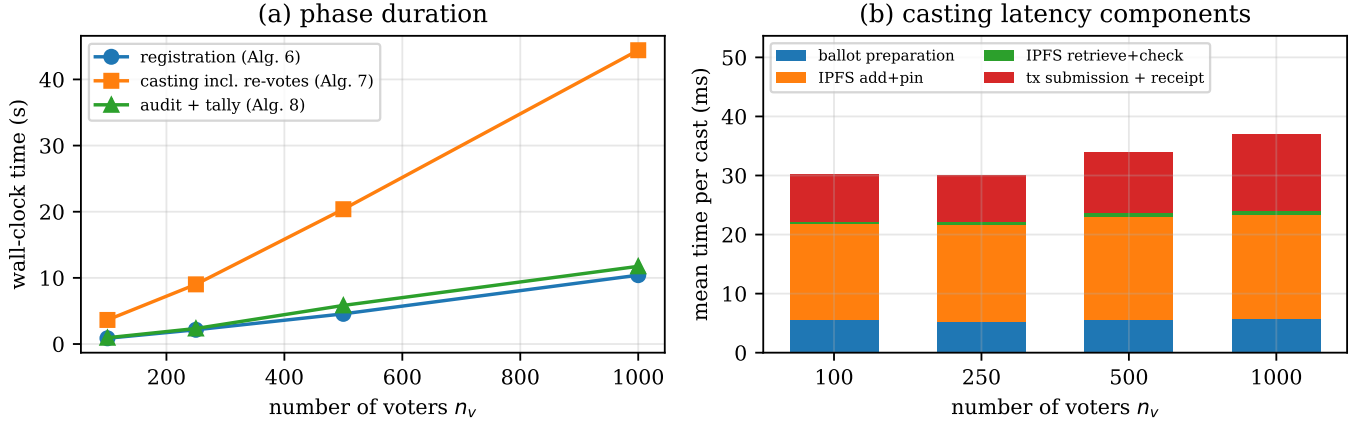


Fig. 5. End-to-end runs. (a) Wall-clock duration of the registration, casting, and audit-and-tally phases versus n_v . (b) Mean latency components of one cast: ballot preparation, local IPFS add and pin, retrieval and comparison, and transaction submission with receipt on the development node.

Prague rules, but it seals each transaction immediately on a single node; IBFT message complexity, validator networking, finality latency, and transaction-pool pre-validation are not measured. (iii) IPFS ran in offline mode on a local node, so the add and retrieve timings exclude wide-area transfer and replication among independent pinning providers. (iv) Real civil-identity authentication, the anonymity-preserving channel of Assumption 1, and the browser environment are outside the measured system. (v) The contract and the reference implementation are research prototypes that have not been independently audited.

G. Supported conclusion

On BLS12-381, the threshold-credential layer adds 0.20 ms of voter CPU time and 0.93 ms of verification time per ballot, and registration with $t_i = 3$ costs the voter 5.91 ms. The complete Helios-derived ballot for five candidates is prepared in 4.83 ms and verified off-chain in 5.72 ms. Enforcing checks C1–C5 in an Ethereum-compatible contract costs about 0.89 million gas per ballot for five candidates, most of which is on-chain proof verification. This cost is feasible for a permissioned ledger with a zero gas price, but it is an order of magnitude above the baseline’s record-only transaction and bounds throughput at about 33 ballots per 30-million-gas block; the per-wallet rate limit prevents a single credential from exhausting this capacity. Complete elections with up to 1,000 voters, including re-voting, produced correct and independently re-verified tallies. The measurements do not establish IBFT latency, wide-area IPFS availability, or user-perceived latency, which remain subjects of future evaluation.

VIII. LIMITATIONS AND DEPLOYMENT ASSUMPTIONS

The following limitations define the scope of the claims.

- **No coercion resistance or receipt freeness.** Wallet control, transaction receipts, and remote voting do not prevent vote buying or coercion. The protocol is unsuitable for high-coercion national elections without a separate coercion-resistant design; combining remote coercion resistance with cast-as-intended verification remains a distinct design problem [44].
- **No software independence.** The Helios challenge improves cast-as-intended assurance, but there is no voter-verifiable physical record independent of the software stack.
- **Communication metadata.** Blind issuance hides the wallet cryptographically; it does not hide IP addresses, timing, browser fingerprints, or wallet-funding traces. Registration-to-wallet unlinkability requires the environmental assumption stated in Assumption 1.
- **Threshold and validator collusion.** A threshold of issuers can authorize additional wallets, a threshold of trustees can violate ballot secrecy, and validators beyond the IBFT fault bound can violate ledger guarantees. Trust is distributed, not eliminated.
- **Registration service and identity provider.** A malicious registration service can deny or delay registration, and with (n_i, t_i) such that $2t_i - n_i \leq f_i$ it can help a corrupted voter obtain more than one credential (Proposition 15). Independent authentication by the issuers protects abstaining voters only if the issuers do not all rely on a

single identity provider.

- **Ledger capacity.** Zero-priced transactions combined with on-chain proof verification make block gas the scarce resource. The rate limit of C5 bounds legitimate wallets, but the protection against invalid transactions relies on validator-side pre-validation, which is not part of the measured prototype.
- **Availability.** IPFS content identifiers authenticate objects but do not keep them online. Because every accepted object is also stored as calldata, audit-data availability can rest on retained ledger history; if validators prune transaction history, multiple independent pinning and archival providers are required. Issuer and trustee liveness depend on their respective thresholds.
- **Registration timing and usability.** Blind threshold issuance is interactive and must be completed before voting or within a sufficiently long election window. This may be unsuitable for elections requiring a single short voter session. No usability study is reported.
- **Wallet recovery and revocation.** The current design permits one credentialed wallet per identity and provides neither secure replacement of a lost or stolen key nor privacy-preserving credential revocation.
- **Implementation evidence.** The evaluation measures CPU cost, object sizes, and exact EVM gas, and it includes complete elections with up to 1,000 voters. It runs on a single-node development chain and an offline local IPFS node, so it does not measure IBFT consensus latency, validator networking, wide-area IPFS retrieval, anonymous-channel overhead, or browser-client performance. On-chain proof verification costs about 0.89 million gas per ballot for five candidates and grows by about 113,800 gas per candidate, which bounds throughput per block.
- **Cryptographic horizon.** The stated instantiation is not post-quantum secure. Parameter choices, serialization, domain separation, and dependency versions must be frozen and independently reviewed before deployment.

These limitations motivate future work on anonymous transport integration, accountable threshold issuance, wallet recovery and revocation, a formally verified contract, replicated archival policy, usability testing, batch verification of the ballot proofs to reduce gas, and benchmarking on a multi-validator IBFT network with wide-area IPFS replication. They also explain why PROBA is positioned as an organizational-election architecture rather than a national-election solution.

IX. CONCLUSION

This paper revisits blockchain-powered Helios at the authorization boundary. The Perez–Ceasay design improves the replication of audit data but assigns wallet eligibility to a contract owner and exposes the identity–wallet association during registration. PROBA replaces the preloaded wallet list with blind threshold credential issuance on an election-specific wallet key, makes every ballot statement explicit and signed, and defines a deterministic latest-valid-ballot rule. A permissioned Ethereum-compatible ledger is treated as one concrete

bulletin-board realization whose guarantees depend on the validator threshold.

The revised analysis separates the resulting properties. Eligibility follows from one-more threshold credential unforgeability, collision resistance of the attribute hash, and independent issuer authentication; registration-to-wallet unlinkability follows from credential blindness and unlinkability under a communication-metadata assumption; ballot privacy follows from Helios encryption, zero knowledge, and simulation-sound extractability of the ballot proofs; and end-to-end verification follows from the ballot challenge, signed content identifiers, deterministic ballot selection, valid-ballot proofs, and verifiable threshold decryption. Robustness and availability are conditional on responsive issuer and trustee thresholds, validator consensus, retained ledger history or replicated IPFS archival, and validator-side pre-validation of transactions.

The new BLS12-381 implementation shows that the credential layer adds 0.20 ms to ballot preparation and 0.93 ms to verification. A five-candidate ballot is prepared in 4.83 ms and verified in 5.72 ms. Enforcing checks C1–C5 on-chain costs about 0.89 million gas per ballot. This is dominated by proof verification and exceeds the baseline’s record-only transaction by roughly an order of magnitude, a cost that a zero-priced permissioned ledger can absorb at about 33 ballots per 30-million-gas block when re-voting is rate-limited per wallet. Complete runs with up to 1,000 voters produced correct tallies. Within these limits, PROBA provides a clearer and more accountable authorization architecture for low-coercion organizational elections while preserving the Helios-derived ballot and tally workflow.

APPENDIX A

CREDENTIAL STATEMENT AND PUBLIC VERIFICATION RELATION

For clarity, the application-specific relations are stated explicitly here. During issuance, the voter proves knowledge of (m_v, o, r) such that $C = g_1^o h_1^{m_v}$ and $D = g_1^r h^{m_v}$ with $h = H_1(C)$. The proof does not show that m_v is the hash of a well-formed wallet key; this is unnecessary, because the verifier recomputes $m_v = H_{\text{attr}}(\text{eid}, \text{pk}_v, T_{\text{cred}})$ from the public wallet key at casting time. During showing, the voter publishes $\pi_v^{\text{cred}} = (h', s')$, and the verifier checks

$$\begin{aligned} \mathcal{TAC}. \text{Verify}(\text{mvk}; \text{eid}, \text{pk}_v, T_{\text{cred}}, \pi_v^{\text{cred}}) &= 1 \\ \iff h' &\neq 1 \wedge e(h', \alpha \beta_2^{m_v}) = e(s', g_2). \end{aligned}$$

The election identifier and wallet public key are disclosed at casting, while the credential signature is re-randomized. The contract separately verifies possession of the corresponding wallet secret key through τ_v , which covers cid_v and hence π_v^{cred} . Thus, a credential cannot be transferred to a different wallet without either forging the wallet signature or obtaining a credential on the replacement key.

DATA AVAILABILITY STATEMENT

The source code and implementation materials are hosted at https://github.com/oguz-yayla/PROBA_benchmark.

The benchmark suite (reference implementation, Solidity contracts, experiment scripts, and table and figure generators), all raw timing samples, and the recorded environment meta-data (hardware, operating system, library, compiler, and node versions) that underlie Section VII are released with the implementation.

REFERENCES

- [1] M. P. Heintz, S. Götz, and C. Bösch, “Remote electronic voting in uncontrolled environments: A classifying survey,” *ACM Computing Surveys*, vol. 55, no. 8, pp. 1–44, 2022.
- [2] B. Adida, O. de Marneffe, O. Pereira, and J.-J. Quisquater, “Electing a university president using open-audit voting: Analysis of real-world use of helios,” in *2009 Electronic Voting Technology Workshop/Workshop on Trustworthy Elections (EVT/WOTE ’09)*. USENIX Association, 2009.
- [3] V. Cortier, P. Gaudry, and S. Glondu, “Belenios: A simple private and verifiable electronic voting system,” in *Foundations of Security, Protocols, and Equational Reasoning*. Springer, 2019, pp. 214–238.
- [4] R. L. Almeida, F. Baiardi, D. D. F. Maesa, and L. Ricci, “Impact of decentralization on electronic voting systems: A systematic literature survey,” *IEEE Access*, vol. 11, pp. 132 389–132 423, 2023.
- [5] Y.-X. Kho, S.-H. Heng, and J.-J. Chin, “A review of cryptographic electronic voting,” *Symmetry*, vol. 14, no. 5, p. 858, 2022.
- [6] J. Benaloh, R. L. Rivest, P. Y. A. Ryan, P. B. Stark, V. Teague, and P. L. Vora, “End-to-end verifiability,” 2015.
- [7] B. Adida, “Helios: Web-based open-audit voting,” in *17th USENIX Security Symposium*. USENIX Association, 2008, pp. 335–348.
- [8] V. Cortier and B. Smyth, “Attacking and fixing helios: An analysis of ballot secrecy,” in *Information Security and Cryptology—ICISC 2015*, ser. Lecture Notes in Computer Science. Springer, 2016, vol. 9558, pp. 236–256.
- [9] M. Meyer and B. Smyth, “Exploiting re-voting in the helios election system,” 2016.
- [10] A. M. Kharman, B. Smyth, and F. Page, “Is your vote truly secret? ballot secrecy iff ballot independence: Proving necessary conditions and analysing case studies,” 2023.
- [11] C. Culnane and S. Schneider, “A peered bulletin board for robust use in verifiable voting systems,” 2014.
- [12] N. Chondros, B. Zhang, T. Zacharias, P. Diamantopoulos, S. Maneas, C. Patsonakis, A. Delis, A. Kiayias, and M. Roussopoulos, “Distributed, end-to-end verifiable, and privacy-preserving internet voting systems,” 2016.
- [13] A. J. Perez and E. N. Ceesay, “Improving end-to-end verifiable voting systems with blockchain technologies,” in *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*. IEEE, 2018, pp. 1108–1115.
- [14] S. Haber, J. Benaloh, and S. Halevi, “The helios e-voting demo for the iacr,” International Association for Cryptologic Research, Tech. Rep., 2010. [Online]. Available: <https://www.iacr.org/elections/eVoting/heliosDemo.pdf>
- [15] Communications of the ACM Staff, “ACM’s 2014 general election: Please take this opportunity to vote,” *Communications of the ACM*, vol. 57, no. 5, pp. 9–17, 2014.
- [16] International Association for Cryptologic Research, “Iacr elections and electronic voting information,” <https://www.iacr.org/elections/>, accessed July 2026.
- [17] Helios Voting Project, “Helios voting,” <https://heliosvoting.org/>, accessed July 2026.
- [18] H. Druliac, M. Bardsley, C. Riches, C. Dunn, and L. Harrison, “On the feasibility of end-to-end verifiable online voting: A case study from the durga puja trial,” *Journal of Information Security and Applications*, vol. 82, p. 103723, 2024.
- [19] J. Müller, “Ordinos: A verifiable tally-hiding e-voting system,” in *2020 IEEE European Symposium on Security and Privacy*. IEEE, 2020.
- [20] D. Gaweł, M. Kosarzewski, P. L. Vora, H. Wu, and F. Zagórski, “Apollo—end-to-end verifiable internet voting with recovery from vote manipulation,” in *Electronic Voting: First International Joint Conference, E-Vote-ID 2016*. Springer, 2017, pp. 125–143.
- [21] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “A homomorphic LWE-based e-voting scheme,” in *Post-Quantum Cryptography—PQCrypto 2016*. Springer, 2016, pp. 245–265.
- [22] Z. Yan, Z. Jiang, and Y. Li, “Towards better privacy-preserving electronic voting system,” 2022.
- [23] E. A. Quaglia and B. Smyth, “A short introduction to secrecy and verifiability for elections,” 2017.
- [24] J. Huang, D. He, M. S. Obaidat, P. Vijayakumar, M. Luo, and K.-K. R. Choo, “The application of the blockchain technology in voting systems: A review,” *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–28, 2021.
- [25] U. Jafar, M. J. A. Aziz, and Z. Shukur, “Blockchain for electronic voting system—review and open research challenges,” *Sensors*, vol. 21, no. 17, p. 5874, 2021.
- [26] M.-V. Vladucu, Z. Dong, J. Medina, and R. Rojas-Cessa, “E-voting meets blockchain: A survey,” *IEEE Access*, vol. 11, pp. 23 293–23 308, 2023.
- [27] A. Emami, H. Yajam, M. A. Akhaee, and R. Asghari, “A scalable decentralized privacy-preserving e-voting system based on zero-knowledge off-chain computations,” *Journal of Information Security and Applications*, vol. 79, p. 103645, 2023.
- [28] C. Onur and A. Yurdakul, “Electanon: A blockchain-based, anonymous, robust and scalable ranked-choice voting protocol,” 2022.
- [29] A. Spanos and I. Kantzavelou, “A blockchain-based electronic voting system: Ethervote,” 2023.
- [30] S. Elnour, W. J. Buchanan, P. Keating, M. Abubakar, and S. Elnour, “vspace: Voting in a scalable, privacy-aware and confidential election,” 2024.
- [31] R. Saltini and D. Hyland-Wood, “Correctness analysis of IBFT,” 2019.
- [32] H. Moniz, “The istanbul BFT consensus algorithm,” 2020.
- [33] A. Faz-Hernández, S. Scott, N. Sullivan, R. S. Wahby, and C. A. Wood, “Hashing to elliptic curves,” RFC 9380, Internet Engineering Task Force, 2023.
- [34] A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, “Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers,” 2018.
- [35] A. Rial and A. M. Piotrowska, “Security analysis of coconut, an attribute-based credential scheme with threshold issuance,” *Cryptology ePrint Archive*, 2022.
- [36] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, “Secure distributed key generation for discrete-log based cryptosystems,” *Journal of Cryptology*, vol. 20, pp. 51–83, 2007.
- [37] T. P. Pedersen, “Non-interactive and information-theoretic secure verifiable secret sharing,” in *Advances in Cryptology—CRYPTO ’91*. Springer, 1991, pp. 129–140.
- [38] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Advances in Cryptology—CRYPTO ’86*. Springer, 1986, pp. 186–194.
- [39] D. Pointcheval and J. Stern, “Security arguments for digital signatures and blind signatures,” *Journal of Cryptology*, vol. 13, pp. 361–396, 2000.
- [40] D. Chaum and T. P. Pedersen, “Wallet databases with observers,” in *Advances in Cryptology—CRYPTO ’92*. Springer, 1993, pp. 89–105.
- [41] D. Bernhard, O. Pereira, and B. Warinschi, “How not to prove yourself: Pitfalls of the Fiat-Shamir heuristic and applications to Helios,” in *Advances in Cryptology—ASIACRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7658. Springer, 2012, pp. 626–643.
- [42] S. Faust, M. Kohlweiss, G. A. Marson, and D. Venturi, “On the non-malleability of the Fiat-Shamir transform,” in *Progress in Cryptology—INDOCRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7668. Springer, 2012, pp. 60–79.
- [43] D. Bernhard, V. Cortier, D. Galindo, O. Pereira, and B. Warinschi, “SoK: A comprehensive analysis of game-based ballot privacy definitions,” in *2015 IEEE Symposium on Security and Privacy*. IEEE, 2015, pp. 499–516.
- [44] T. Finogina and J. Herranz, “On remote electronic voting with both coercion resistance and cast-as-intended verifiability,” *Journal of Information Security and Applications*, vol. 76, p. 103554, 2023.